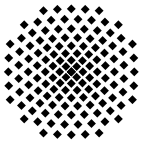


University of Stuttgart
Institute for Signal Processing and System Theory
Professor Dr.-Ing. B. Yang

German International University
Media Engineering and Technology
Professor Dr.-Ing T. Ince

**Bachelorarbeit S1509**

# Implementation of an Electromyography Processing Pipeline from Torch model to TinyML on resource-constrained microcontroller platforms

**Implementierung einer Elektromyographie-Verarbeitungspipeline von einem Torch-Modell zu TinyML auf ressourcenbeschränkten Mikrocontroller-Plattformen**

Author: Andrew Fady Fahmy

Date of work begin: 01/03/2026
Date of submission: 30/06/2026

Supervisor: Jonas Koellner, Prof. Dr-Ing Turker Ince

Keywords: sEMG, TinyML, Digital Signal Processing, Deep Learning, regression, Limited-Resource Microcontroller Platforms

This thesis investigates the deployment of a convolutional–recurrent surface electromyography (sEMG) gesture-regression model developed by Koellner and Yang on a commodity, resource-constrained microcontroller (the ESP32-S3) using a portable, non-destructive methodology: a pre-trained PyTorch network is adapted for embedded inference via the industry-standard LiteRT framework without retraining. A modular pipeline partitions the network into four exportable stages and exports the LSTM as a windowed block, reducing the memory footprint from over $2.8MB$ to about $260KB$ and enabling mixed-precision deployment that roughly halves the latency of the default floating-point model. Results show int8 quantization is highly effective for the convolutional stages with varying accuracy across trials but counterproductive for the recurrent stages, where requantization dominates latency and error compounds across timesteps, thus characterising both the promise and the limits of portable, framework-based TinyML for recurrent biosignal models.

# Contents

# 1 Introduction

Human–machine interaction (HMI) aims towards interfaces that feel natural rather than mechanical, in which a system responds to a person's intended movement as fluidly as a limb responds to its owner [1, 2]. Hand gestures are central to this goal: beyond enabling more natural and accessible communication for users of diverse abilities, they underpin prosthetic and rehabilitation devices that allow individuals with limb differences or impairments to interact with technology seamlessly, and the ability to interpret and reproduce natural hand movement is likewise essential to tools that support the recovery of motor skills after injury or illness [3]. Among the available control modalities, the decoding of hand and wrist gestures from surface electromyography (sEMG) is especially compelling, because it reads the electrical intent of muscle activation directly and so offers a more immediate channel of communication than buttons or touchscreens. It also holds a practical advantage over camera-based approaches: vision systems depend on controlled illumination and an unobstructed line of sight for reliable performance, and typically rely on more complex machine-learning pipelines, whereas a body-worn sEMG sensor is unaffected by lighting or occlusion and captures muscle intent at its source [3].

Two applications make this capability particularly consequential. The first is immersive interaction: gesture decoding enables natural, hands-free control within augmented- and virtual-reality environments and other wearable systems, where continuous estimation of movement is more expressive than the recognition of a fixed set of discrete poses. The second, and more socially significant, is assistive technology: for individuals living with amputation or limb difference, an sEMG interface that continuously and proportionally estimates intended movement is the foundation of a myoelectric prosthesis that can be controlled smoothly and intuitively, restoring a measure of natural function and independence.

Despite these applications, such systems remain far from widely available. Accurate gesture decoding has historically depended either on costly, specialized hardware or on offloading computation to remote servers, an arrangement that raises several barriers for everyday users [4]. Hardware that is powerful enough to run modern deep-learning models is expensive, placing capable prosthetic and HMI devices beyond the reach of many who would benefit most from them. The alternative, cloud-based inference, lowers the on-device hardware requirement but ties the user to continuous network connectivity, introduces communication latency that is incompatible with real-time control, and transmits sensitive physiological data off the device, raising privacy and reliability concerns that are detrimental to medical and assistive applications [5, 6]. In both cases the result is the same: a technology of considerable human value that is, in practice, inaccessible, costly, or dependent on a system that cannot be guaranteed.

This motivates doing gesture-decoding inference onto low-cost, self-contained microcontroller hardware. A microcontroller-based implementation removes the dependence on remote servers and the connectivity it requires and keeps physiological data local to the device without the need expensive accelerators, thereby addressing all of the concerns above. However, deployment of deep learning pipelines on resource constrained systems is not straightforward, and it is precisely this deployment challenge that the present thesis addresses through the field of TinyML.

This thesis investigates the deployment of a convolutional–recurrent sEMG gesture-regression model on a commodity and a widely available resource-constrained microcontroller, the ESP32-S3, using a portable, non-destructive methodology. A pre-trained PyTorch network is adapted for embedded inference through the industry-standard LiteRT framework without retraining or altering its architecture. To this end, a modular pipeline partitions the network into independently exportable stages, exports the LSTM as a windowed block invoked repeatedly on the device, and applies a series of graph-level transformations to obtain efficient quantized stages, enabling a mixed-precision deployment in which integer and floating-point stages operate alongside one another.

The work is organized as follows. Chapter 2 establishes the necessary background, covering bioelectric signals and sEMG, the classical and deep-learning approaches to gesture decoding, the constraints of microcontroller platforms, the TinyML paradigm, and the principles of model quantization. Chapter 3 surveys the state of the art in embedded biosignal processing, drawing on both ECG and sEMG work, and identifies the specific gaps in the literature that motivate this work. Chapter 4 describes the materials and methods: the NinaPro DB2 dataset and its preprocessing, the deep-learning architecture, the ESP32-S3 hardware platform, the quantization scheme, the software tools, and the modular deployment pipeline together with the graph-level transformations it required. Chapter 5 presents the experimental results, encompassing an operation-level benchmark of model primitives, a computational-complexity analysis, and a comparison of complete model configurations. Chapter 6 discusses these results in depth, and Chapter 7 concludes the thesis and outlines directions for future work.

# 2 Basics and Foundations

## 2.1 EMG and Deep Learning Pipelines for Bioelectric Signal Processing

### 2.1.1 Bioelectric Signals and Human Machine Interaction (HMI)

A bioelectric signal originates from the human body; it is the sum of the action potentials of cells at an anatomical site such as the heart, the brain, or skeletal muscle [7]. Such signals have been used extensively in Human–Machine Interaction (HMI), as they provide key insights into a person's physiological activity and offer a more natural medium of interaction between humans and computers. Although the primary focus of this work is the Electromyogram (EMG), we frequently refer to other commonly studied bioelectric signals, namely the Electroencephalogram (EEG) and the Electrocardiogram (ECG), given the wealth of processing pipelines and HMI research developed in these two domains.

### 2.1.2 Electromyography (EMG) Basics

Electromyography is the recording of the electrical activity of muscle. The underlying electrical impulses originate in the spinal cord and propagate through the peripheral nerves to activate muscular contraction [1]. A motor neuron together with the muscle fibres it innervates is termed a motor unit; motor units are recruited by these impulses and, in doing so, produce electrical signals. Professional electromyographers analyse characteristic features of these signals, such as their waveforms, firing rates, and the frequency of abrupt discharges, to derive diagnostic information for identifying abnormalities and various neural and muscular diseases.

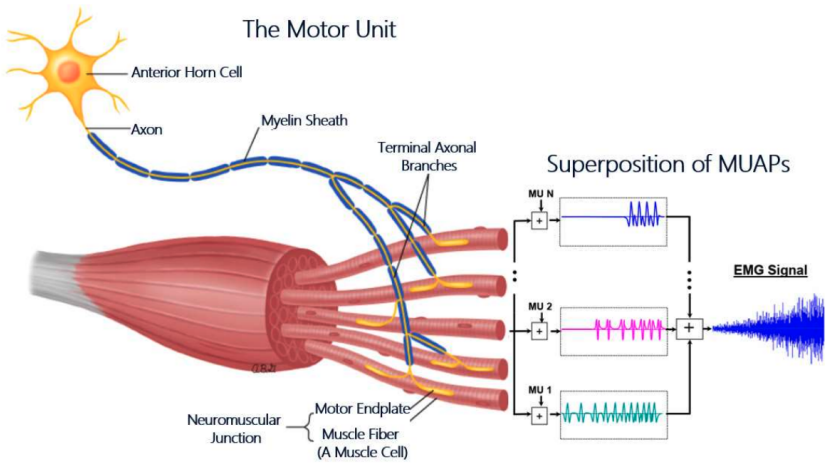



Figure 2.1: The diagram shows the origin of the EMG signal [1].

The recording of EMG signals is broadly divided into two categories. The first is intramuscular electromyography (iEMG), an invasive technique in which concentric needle electrodes are inserted directly into the muscle. The second is surface electromyography (sEMG), a non-invasive technique in which adhesive electrodes are placed on the surface of the skin. While iEMG has proven more effective for diagnosing medical conditions and abnormalities, sEMG dominates the HMI and gesture-recognition sector owing to its non-invasive nature, its wider accessibility without the need for trained experts, and the greater comfort it affords the user [2].

For these reasons, this study and [8] focused exclusively on sEMG data.

### 2.1.3 EMG Signal Preprocessing and Feature Extraction

Raw EMG signals suffer from a multitude of issues that obscure their insightful value. EMG signals are susceptible to various types of noise, which include electrical interference, physiological signals from adjacent muscles that can contaminate the signal, and the inherent instability of the signal, which arises because the firing rate of the motor units is itself quasi-random [9]. Relevant information is further obscured by movement artifacts and by variability from one subject to another [2, 10].

These problems clearly motivate extensive pre-processing in order to recover as much insightful information as possible from EMG signals [11]. Typically, sEMG signals recorded by commonly available sensors lie in the range of 0 to 400 Hz. Fourth-order Butterworth band-pass filters with a 20–450 Hz passband are used by most studies to retain viable information while discarding the 5–20 Hz corner frequencies [11]. To suppress undesired high-frequency noise, many studies additionally recommend smoothing techniques such as forward–backward filtering. Normalisation is likewise ubiquitous in digital signal processing and machine learning, as it promotes stable and faster training while reducing amplitude variability across subjects, sessions, and electrode placements [10].

Windowing is also widely adopted in EMG pre-processing pipelines, with parameter choices varying across the literature. For real-time closed-loop control: a maximum latency of 300 ms was initially recommended [12], although more recent studies suggest that it is optimally kept between 100 and 250 ms [10, 13]. While larger window sizes are preferred for higher-quality, more accurate models, smaller window sizes are the main choice for real-time and edge solutions that prioritise speed under limited resources.

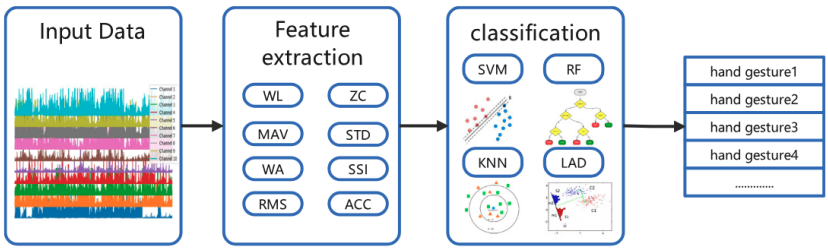
Beyond filtering and windowing, classical pipelines incorporate a further pre-processing stage, namely *feature extraction*, which is emphasised across both research papers and textbooks. In digital signal processing, the extracted features fall into two main categories. Time-domain features are computed directly from the filtered time series, and include the mean absolute value (MAV), waveform length (WL), number of zero crossings (ZC), and root mean square (RMS); these are widely adopted, with each contributing to recognition performance to varying degrees. Frequency-domain features, by contrast, are derived from the Fourier transform of the time series and include measures such as the mean frequency (MNF) and peak frequency (PKF) [14].

It is important to emphasise, however, that such hand-crafted features are characteristic of the classical machine learning paradigm: they are designed to transform the raw signal into a compact, discriminative representation suitable for classifiers such as the SVM. Deep learning models, by contrast, are designed to operate end-to-end, learning task-relevant features directly from the filtered signal rather than relying on features specified in advance by a human expert [15].

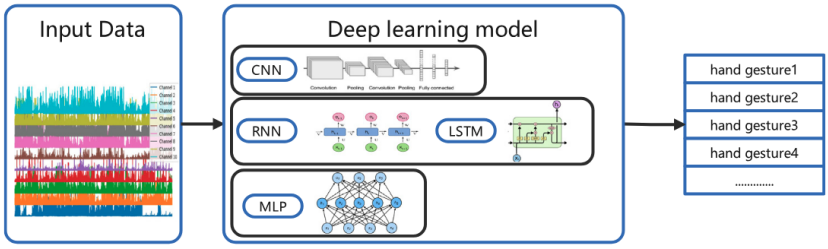
Accordingly, the pipeline deployed in this thesis does not employ any hand-crafted feature extraction. Pre-processing is restricted to the time domain, filtering, windowing, and normalisation, after which the deep learning model performs its own feature extraction internally. This choice reflects the broader shift in the field away from manual feature engineering and toward feature learning [15].

### 2.1.4 Machine Learning and Deep Learning Approaches for sEMG-Based Classification and Regression

Approaches to sEMG decoding fall broadly into two paradigms: classical machine learning (ML) methods, which operate on hand-crafted features extracted from the signal, and deep learning (DL) methods, which learn task-relevant representations directly from the raw or minimally processed signal. This section surveys both, establishing the context for the resource-constrained deployment that this thesis addresses.




(a) Traditional machine learning methods




(b) Deep learning methods

Figure 2.2: The process of ML based pipelines vs DL based pipelines from [3].

#### Classical Machine Learning Approaches

Among classical ML classifiers, the Support Vector Machine (SVM) is one of the most widely adopted methods for sEMG-based gesture recognition [3]. SVMs have been applied extensively to sEMG classification, with feature-augmented variants reporting average accuracies as high as 99.98% on constrained gesture sets [16]. The method is regarded as one of the more robust and accurate classical classifiers for sEMG [17], and is frequently employed as the reference baseline against which newly proposed sEMG decoders, including deep models intended for embedded deployment, are evaluated [18]. Other classical classifiers recurrently reported in the literature include Linear Discriminant Analysis (LDA), $k$-Nearest Neighbours (kNN), and Random Forests [3], with comparative studies often finding LDA and SVM to yield broadly comparable accuracies on time-domain feature sets.

The principal limitation shared by these classical methods is their dependence on a hand-crafted feature extraction and selection pipeline. Such pipelines require domain expertise to design, introduce subjectivity in the choice of features and parameters, and must frequently be re-tuned when signal characteristics change, which constrains their flexibility and generalisability [3].

#### Deep Learning Approaches

Deep learning circumvents the manual feature-engineering bottleneck by learning hierarchical, task-relevant representations directly from the data, enabling end-to-end classification and regression [19]. Owing to its successes across domains ranging from computer vision to digital signal processing, broad methods of DL architectures have been adapted to sEMG decoding. Crucially, the choice of architecture is tightly coupled to the representation in which the sEMG signal is cast [19].

When the multi-channel sEMG is arranged spatially, treating electrode channels as a spatial axis, it can be processed as an image, making Convolutional Neural Networks (CNNs) a natural and widely used choice, mirroring their dominance in image analysis [19]. When instead the signal is treated as a set of temporal waveforms, sequence-modelling architectures become appropriate. Recurrent Neural Networks (RNNs) capture temporal dependencies by processing the sequence step by step, while their gated variants—the Long Short-Term Memory (LSTM) and the Gated Recurrent Unit (GRU), introduce memory cells and gating mechanisms that mitigate the vanishing and exploding gradient problems, thereby improving the modelling of long-range temporal dependencies [19].

Temporal Convolutional Networks (TCNs) have more recently emerged as a compelling alternative to recurrent models for sEMG sequence modelling. Because they apply convolutions along the time axis, TCNs process entire segments in parallel rather than sequentially, affording greater training efficiency and scalability than RNNs [19]. Through dilated causal convolutions, a TCN can expand its receptive field, and thus the temporal context it captures, exponentially with depth, without a significant increase in parameter count [18, 19]. These properties have made TCNs particularly attractive for real-time, resource-constrained sEMG decoding, a point developed further in the subsequent discussion of embedded and TinyML solutions.

## 2.2 TinyML and Resource-Constrained Microcontroller Platforms

### 2.2.1 Microcontrollers

A microcontroller unit (MCU) is a self-contained computing system integrated onto a single chip. It comprises the core components required for general-purpose computation, including a central processing unit (CPU), volatile memory (RAM), non-volatile FLASH storage, and input/output (I/O) peripherals. The adoption of MCUs has accelerated markedly in recent years, driven largely by the spread of IoT systems (Internet of Things) [6], which deploy numerous interconnected devices to acquire, process, and extract actionable insights from on-ground sensor data, transmitting this information across dedicated channels and networks for subsequent processing and decision-making.

The contemporary MCU market is served by several prominent vendors, including Arduino (notably the UNO and Nano modules), Espressif Systems (the ESP32 family), and STMicroelectronics (the STM32 series), among others. This landscape continues to evolve rapidly as new architectures and capabilities emerge.

### 2.2.2 Limitations and Constraints of Microcontroller Platforms

The low cost, compact form factor, and minimal power consumption that characterise MCUs are achieved through significant trade-offs relative to general-purpose and high-performance computing platforms.

Foremost among these is severely constrained memory [6]. On-chip memory is approximately three orders of magnitude smaller than that of mobile devices, and five to six orders of magnitude smaller than that of cloud-based GPUs [20]. Clock frequencies are similarly limited, typically operating in the megahertz (MHz) range, in contrast to the gigahertz (GHz) range of modern general-purpose processors [5]. A further constraint is the absence of GPUs or dedicated parallel processing units, which restricts the efficient execution of single-instruction-multiple-data (SIMD) operations and vectorised computation; this limitation has historically impeded the adoption of MCUs in the graphics and machine learning domains [5]. Moreover, software development for MCUs remains notoriously cumbersome, owing to fragmented vendor-specific SDKs and frameworks, the prevalent reliance on deprecated software to accommodate hardware constraints, and the limited continuity of support and maintenance for development toolchains.

Table 2.1: Comparison of commercially available MCUs for edge ML applications.

| MCU | Processor | WiFi/BT | Memory & FPU | NN / SIMD | Cost |
|---|---|---|---|---|---|
| ESP32 | Xtensa LX6, 2× @ 240 MHz | Yes | 520 KB SRAM; 8 MB PSRAM + 16 MB Flash (ext.); SP FPU | ESP-DL / ESP-NN | $3–8 |
| ESP32-S3 | Xtensa LX7, 2× @ 240 MHz | Yes | 512 KB SRAM; 8 MB PSRAM + 16 MB Flash (ext.); SP FPU | ESP-DL / ESP-NN (vector ext.) | $4–10 |
| Pi Pico (RP2040) | Cortex-M0+, 2× @ 133 MHz | Wi-Fi[a] | 264 KB SRAM; ext. PSRAM; 2 MB Flash; no FPU | CMSIS-NN (no SIMD) | $4–6 |
| Arduino Nano | ATmega328P (8-bit), 1× @ 16 MHz | No | 2 KB SRAM; no PSRAM; 32 KB Flash; no FPU | None | $10–25 |
| STM32H7 | Cortex-M7, 1–2× @ 480 MHz | No | 1 MB SRAM; ext. PSRAM; 2 MB Flash; DP FPU | CMSIS-NN/DSP (no MVE) | $8–20 |
| STM32N6 | Cortex-M55 + Neural-ART NPU, 1× @ 800 MHz | No | 4.2 MB SRAM; ext. PSRAM; flashless[b]; H/SP/DP FPU | Helium SIMD + NPU (600 GOPS); ST Edge AI | $5–12 |

[a] Wi-Fi on Pico W only; base RP2040 has no radio. [b] No internal user flash; boots from external XSPI. NPU clocked at 1 GHz.

### 2.2.3 TinyML

TinyML constitutes a subfield of Edge ML concerned specifically with deploying deep learning models on low-cost, highly resource-constrained MCUs through the joint optimisation of ML kernels and the underlying hardware to satisfy the prevailing constraints. Since its emergence, TinyML has gained considerable traction and been adopted across numerous sectors that previously depended on Cloud ML.

The first widely known application was simple wake-word detection for Amazon, Apple, and Google [21]; since then, the range of applications has grown significantly. In the healthcare domain, this includes extensive use of wearable technology for low-cost health monitoring, which demands the reliable and secure processing of sensitive personal data alongside the rapid and timely detection of critical health abnormalities and emergency conditions [6, 5]. Furthermore, owing to the proximity of MCUs to sensory data and their capacity to respond with minimal latency, TinyML is well suited to gesture-based human–machine interaction (HMI). Such applications range from natural and cost-effective real-time interaction within AR/VR environments to more consequential uses, such as facilitating more intuitive control of prosthetic limbs for individuals with disabilities and amputations [20].

The suitability of TinyML for these applications is best understood by contrast with the approach it displaced. Historically, the only viable means of machine learning (ML) and deep learning (DL) inference within IoT systems was Cloud ML. This paradigm leveraged the communication capabilities of MCUs, such as Wi-Fi, to transmit raw sensor data to remote servers possessing substantially greater computational resources for processing, subsequently retrieving the resulting commands in order to actuate physical responses [20]. While this strategy exploited the full capabilities of cloud clusters and relaxed the hardware requirements of IoT devices, it suffers from several limitations that are particularly acute for the real-time biosignal applications described above, such as the sEMG-based gesture decoding addressed in this work [6].

First, the round-trip transmission of data and commands to and from remote servers introduces latency that is incompatible with the sub-200 ms response times required for natural, real-time prosthetic and human–machine interaction. Second, cloud inference presupposes reliable network connectivity, an assumption that fails for wearable and assistive devices intended to operate continuously and in arbitrary environments [5]. Third, the transmission of raw physiological signals over a network exposes sensitive medical data to security vulnerabilities and privacy violations, a concern that is especially pronounced in healthcare applications. Finally, the Cloud ML paradigm necessitates the development and maintenance of potentially costly server infrastructure, undermining the low-cost, self-contained operation that makes wearable assistive technology widely accessible.

By performing inference directly on the device, TinyML overcomes these limitations: it eliminates network round-trip latency, removes the dependence on connectivity, keeps sensitive physiological data local, and avoids the cost of server infrastructure, which is precisely what makes the wearable, real-time applications described above feasible.

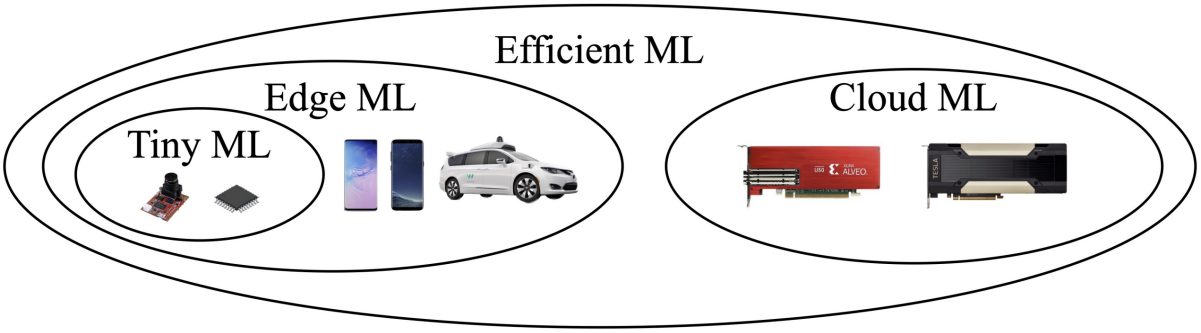



Figure 2.3: Figure from [5] showing the taxonomy of the TinyML field.

## 2.3 Quantization

Normally, Deep learning pipelines rely on floating point operations and embed their parameters (i.e weights and biases) as float32s. Unfortunately, not all MCU architectures have embedded FPU (floating point units) to handle calculations in floating point notations. In addition, chips that have dedicated FPUs in their architecture, for handling float32 data, suffer from longer processing times and resource intensive operation compared to the faster integer arithmetic units [22, 23, 24].Furthermore, floating point numbers necessitate larger spaces in memory for tensor allocations (i.e inputs, outputs, model parameters and activations) [22].
For these reasons, different quantization schemes are widely adopted across TinyML literature It is also the case on many MCU platforms (like the ESP32 and ESP32s3), that optimized low-level kernels for ML ops (like Convolutions and MatMuls) are exclusively written with quantization schemes, like INT8, in mind. In fact, 8-bit quantization has been shown to reduce model sizes by a factor of 4 and produce a speedup gain of 2x-3x ,even 10x on specialized processors, compared to float Implementations[22].

On the other hand, quantization algorithms are not lossless since they map a continuous floating point representation into a discrete range. A degradation in final accuracy is always expected in quantized outputs. That's why quantization pipelines require careful fine-tuning to balance quality and performance.

### 2.3.1 Quantization Types and Schemes

Quantization schemes have been categorized into many sections. In this work, we are going to outline schemes that are only relevant to TinyML research.

#### Data Types

By default, popular ML frameworks like PyTorch, Tensorflow and JAX utilize single-precision floating point numbers (float32s). Quantization schemes can then be categorized into the output data types that these float32s are converted to.

**Float16** quantization uses half the space used by float32s while maintaining the floating

point interface as float32s. However, float16 adoption in MCU architectures still remain rare. Most MCUs don't have dedicated kernels for handling float16s and just use float32 interfaces rendering float16 optimizations completely useless. Others are unable to handle float16 data and immediately throw exceptions.

**Int8, Int16, Int32** quantization are by far the most used. Most MCU architectures are optimized for handling integer operations which makes int quantization adoption easier and more natural. Low-Level kernels usually use a mix of various integer sizes in their internal Implementation, usually inputs, outputs and weights are int8 while biases use larger formats like int32s.

### Asymmetric vs symmetric quantization

**symmetric** quantization restricts the zero-point to 0. It only computes the scale of the floating-point values as the ratio between the absolute maximum value in the floating point range and the maximum value in the quantization range. Commonly used for quantizing weights and biases as[22]. The reason why it is preferred for weights is because using asymmetric quantization (with an additional zero-point value for weights) adds an additional term to each **W . x** calculation which costs additional compute time during inference [24].

$$
\begin{aligned}
x_{int} &= \text{round}\left(\frac{x}{\Delta}\right) \\
x_Q &= \text{clamp}(-N_{levels}/2,\ N_{levels}/2 - 1,\ x_{int}) && \text{if signed} \\
x_Q &= \text{clamp}(0,\ N_{levels} - 1,\ x_{int}) && \text{if un-signed} \\
x_{out} &= x_Q\,\Delta && \text{(de-quantization)}
\end{aligned}
\tag{2.1}
$$

**Asymmetric** quantization calculates an optimal new zero-point value to map the floating point zero to it. Preferred for activations.

### Quantization Aware Training (QAT) vs Post Training Quantization (PTQ)

$$
\begin{aligned}
x_{int} &= \text{round}\left(\frac{x}{\Delta}\right) + z \\
x_Q &= \text{clamp}(0,\ N_{levels} - 1,\ x_{int}) \\
x_{float} &= (x_Q - z)\,\Delta && \text{(de-quantization)}
\end{aligned}
\tag{2.2}
$$

**Post Training Quantization (PTQ)** works on quantization pre-trained DL models, trained using default floating-point numbers. The quantizing algorithm takes pre-trained weights and biases and calculates the appropriate scaling and shifting parameters. For activations, it uses a representative dataset (representing a sample of input values in floating space) and propagates it through the model to get a closer statistical model of how values fluctuate in the actual model in order to map it to the quantized range. PTQs are widely adopted in pipelines that aim at deploying normal, pre-trained models, usually not co-designed for TinyML use, on Edge device without having to re-train the model [22, 24].

**Quantization Aware Training (QAT)** involves having quantization in mind during the training of the actual model, with simulated quantization applied at every step of gradient

descent so that the model learns to compensate for quantization noise. The weights are maintained in floating point and updated using floating-point gradients, while a simulated quantization operation (fake quantization) emulates the effect of integer inference during the forward and backward passes. While this method can provide higher accuracies than PTQ [22, 24], it necessitates active co-design of the DL model and is not viable for quantizing all available models, since it requires retraining in the presence of simulated quantization, which demands additional training resources and time-consuming retraining.

$$\begin{aligned} w_{float} &= w_{float} - \eta \frac{\partial L}{\partial w_{out}} \cdot I_{w_{out} \in (w_{min}, w_{max})} \\ w_{out} &= \text{SimQuant}(w_{float}) \end{aligned} \tag{2.3}$$

### Hybrid vs Full quantization

**Hybrid** quantization is defined as quantizing the inputs, weights and biases only while leaving intermediate activations as floating point. While this can improve final accuracy since a full quantization is not adopted and activations remain in their full floating point resolution, many MCU platforms and frameworks do not fully support mixed precision computation. In addition, Hybrid models produced the overhead of having to requantize activations after each neural layer which can detrimentally impact the final inference time and the needed allocation space for larger float32 intermediate values.

**Full** quantization involves the quantization of the entire Pipeline eliminating the need for intermittent requantization steps and produces faster and more lightweight models since there are no intermediate floats. However, for full quantization, more overhead is needed for calculating additional quantization parameters for activation layers and , depending on the quantization scheme used, may require a set of representative data to calculate those quantization parameters.

### Quantization granularity

Quantization can be done with varying degrees of resolution at the cost of producing an increasing amount of quantization parameters and settings.

**Per Tensor** quantization involves calculating scaling and shifting params for each tensor involved in the pipeline. This scheme is preferred for activations [22].

**Per Channel** quantization involves having a set of quantization params per each channel in a tensor. This presents a more granular quantization scheme as it accounts for distribution differences between individual channels in the data at hand. This is the recommended method for stored weights [22, 24].

# 3 State of The Art (SoTA)

It is generally difficult to assemble a clear SoTA metric given the literature sparsity on Time-Series processing on Resource-Constrained Microcontrollers especially on specific types like EMG. So in order to identify a clear SoTA body, we have to look for other studies on similar Time-Series data structures and other Bioelectric signals ML based processing on microcontrollers.

## 3.1 ECG Classification using TinyML

Owing to the scarcity of prior work on sEMG processing under TinyML constraints, we extend our review to the classification of the electrocardiogram (ECG), a bioelectric signal that shares many characteristics with the EMG and whose embedded deployment has been studied more extensively.

In the context of ECG arrhythmia detection, the study conducted by Faraone et al. [25] employs a convolutional–recurrent architecture closely related to our own. The model comprises seven one-dimensional convolutional layers with a kernel size of five, each followed by an average-pooling layer of size and stride two. The convolutional stage produces a 128-dimensional feature tensor per window, which is subsequently processed by a Gated Recurrent Unit (GRU) with 64 hidden units serving as the recurrent module.

Notably, the implementation adopts a mixed-precision quantization scheme: the convolutional, pooling, and dense layers are quantized to 8-bit fixed point (Q2.5 format), whereas the GRU gates and internal states are retained at 16-bit precision (Q2.13 format) to preserve the fidelity of the recurrent computation [25]. The model is deployed on the nRF52832 System-on-Chip from Nordic Semiconductor, an ARM Cortex-M4 platform, using the CMSIS-NN kernel library.

The network was trained and evaluated on a dataset of 8,528 single-lead ECG recordings sampled at 300 Hz with durations between 9 and 60 seconds. On this four-class classification task, the full-precision model attains a test-set accuracy of 86.1%, which decreases only marginally to 85.4% following fixed-point quantization, with the overall F1 score falling from 0.80 to 0.784. The weights, biases, and lookup tables occupy 195.6 KB of storage, while the complete output binary (including the CMSIS-DSP and CMSIS-NN routines) amounts to approximately 210 KB.

| Layer | Output shape | Parameter count |
|---|---|---|
| Input | ($N_w$, 256, 1) | - |
| Conv1 | ($N_w$, 128, 8) | 48 |
| Conv2 | ($N_w$, 64, 16) | 656 |
| Conv3 | ($N_w$, 32, 32) | 2592 |
| Conv4 | ($N_w$, 16, 64) | 10,304 |
| Conv5 | ($N_w$, 8, 64) | 20,544 |
| Conv6 | ($N_w$, 4, 128) | 41,088 |
| Conv7 | ($N_w$, 2, 128) | 82,048 |
| Global Average Pooling | ($N_w$, 128) | 0 |
| GRU | (64) | 37,056 |
| Dense+Softmax | (4) | 260 |

Figure 3.1: Convolutional–recurrent architecture employed by Faraone et al. [25].

Despite presenting a promising embedded implementation, the approach is subject to several limitations that are directly relevant to our work. First, it operates on comparatively short inputs, processing windows of only 256 samples and a recurrent sequence of just 128 timesteps, which constrains the temporal context available to the model. Second, the recurrent complexity is deliberately curtailed: the authors substitute the LSTM of the original architecture with a less expressive GRU specifically to reduce the memory footprint and operation count. Finally, and most significantly, the recurrent component could not be executed using a general-purpose inference engine; the authors report that TensorFlow Lite for Microcontrollers did not support optimized recurrent operators at the time, necessitating a hand-written CMSIS-NN implementation.

A second representative study is the work of Kim et al. [26], which develops a TinyML-based classifier for an ECG monitoring embedded system (TinyCES). In contrast to the preceding work, the authors adopt a purely convolutional–fully-connected architecture for the binary classification of normal and abnormal ECG segments, comprising two convolutional layers (with 8 and 16 filters of size 4×1, respectively), each followed by max-pooling and dropout, and terminating in two dense layers.

The model is implemented using TensorFlow Lite for Microcontrollers and deployed on an Arduino Nano 33 Bluetooth Low Energy (BLE) Sense board, also based on an ARM Cortex-M4 core. Operating on an input window of 180 samples, the converted model occupies a footprint of only 27 KB and achieves a detection accuracy of approximately 97%.

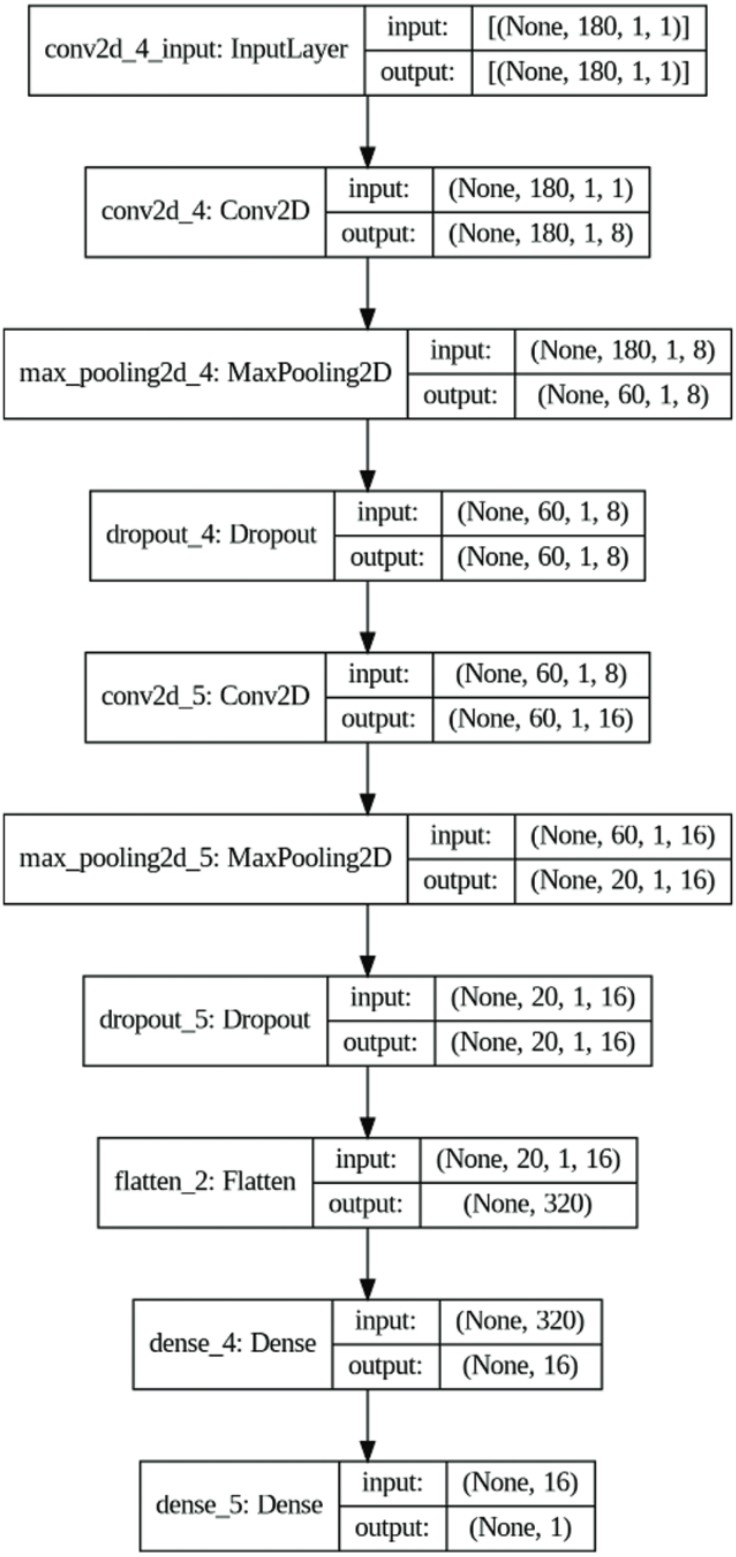



Figure 3.2: Convolutional–fully-connected architecture employed by Kim et al. [26].

The principal limitation of this study lies in the simplicity of its architecture relative to the demands of continuous biosignal regression. The model contains no recurrent component whatsoever, relying solely on convolutional feature extraction followed by dense classification, and

it therefore forgoes any explicit modelling of temporal dependencies across the signal. Furthermore, the comparatively small input window of 180 samples and the reduction of the task to binary classification yield a markedly smaller and less complex model than is required for the multi-channel, multi-output gesture regression addressed in this thesis. Consequently, while the work convincingly demonstrates the feasibility of fully on-device ECG classification, it does not confront the memory and latency challenges posed by recurrent architectures on resource-constrained MCUs.

## 3.2 EMG Signal Processing

The closest prior work to this thesis is a pair of studies conducted by the same group at the University of Bologna, which together address the complete sEMG processing pipeline, from model design through to embedded implementation, on a multicore IoT processor. The first study targets hand-movement *classification* [18], while the second generalises the same methodology to continuous *regression* of hand kinematics [27]. As this thesis is concerned specifically with the implementation side of an analogous pipeline these works constitute the principal point of comparison.

In the classification study [18], the authors deploy TEMPONet, a dilated Temporal Convolutional Network (TCN), on the 8-core GAP8 processor. The architecture stacks three convolutional blocks, each comprising two dilated causal convolutions of kernel size 3×1 (with full padding and increasing dilation) followed by a strided convolution of kernel size 5×1 and an average pooling layer; the convolutional stage is terminated by two fully connected layers with dropout and a SoftMax operation. All layers employ ReLU activations and Batch Normalisation [18]. The model is benchmarked on the multi-session NinaPro DB6 dataset, operating on short 150 ms input windows in keeping with the 300 ms real-time consensus for hand control.

On the implementation side, the PyTorch model is deployed using the dedicated PULP-NN kernel libraries, which exploit the eight RISC-V cluster cores of the GAP8 together with their SIMD extensions. Since the GAP8 lacks a floating-point unit, inference is carried out entirely in 8-bit fixed-point arithmetic. To accommodate the tight on-chip memory, an automated tiling tool (DORY) partitions the weights and feature maps of each layer into tiles that fit the 64 kB L1 scratchpad and inserts double-buffered DMA transfers between the 512 kB L2 memory and L1 so that data movement overlaps with computation [18].

The classification model attains an inter-session accuracy of 49.6% on the full NinaPro DB6 (7.8 percentage points above the prior state of the art) and 93.7% on a 20-session dataset collected by the authors as a proxy for the final deployment. Following int8 quantisation, the network occupies a memory footprint of 460 kB and executes a single inference in 12.84 ms on the GAP8 [18].

The companion regression study [27] retains the same TCN paradigm but adapts and further optimises TEMPONet to estimate five continuous degrees of actuation, evaluating it on NinaPro DB8 (which includes two trans-radial amputees). Through channel reduction and int8 quantisation, the authors reduce the memory footprint to 70.9 kB and the inference latency to 4.76 ms on the GAP8, achieving a mean absolute error of 6.89°, marginally below the float32 LSTM baseline of the dataset [27]. Notably, the authors deliberately favour a purely convolutional model over recurrent alternatives, citing evidence that TCNs match or exceed the accuracy of

LSTMs on sEMG sequence modelling while being substantially more open to parallelisation and embedded deployment.

While both studies represent a compelling state of the art, their design choices differ from those of the present work in three respects that are directly relevant to our contribution. First, with regard to model complexity, both rely on a feature-extraction-only convolutional backbone and explicitly avoid any recurrent component; by contrast, the model deployed in this thesis combines spatial and temporal convolutions, residual concatenation, batch normalisation and ELU activations, a downsampling stage, a two-layer LSTM, and a dedicated regression head. The presence of a genuine recurrent module is significant, as recurrent layers are markedly more difficult to deploy on resource-constrained hardware, precisely the implementation challenge this thesis addresses. Second, with regard to the hardware platform, both works target the GAP8, an 8-core parallel processor with ISA extensions purpose-built for embedded neural-network inference, whereas this work targets the single-core ESP32-S3, a substantially more constrained but far cheaper and more widely available commodity microcontroller. Third, and most consequentially, their deployment depends on a bespoke, platform-specific toolchain (PULP-NN kernels with DORY-based tiling) tightly coupled to the GAP8 architecture, rather than on a portable, industry-standard inference framework. The present work instead builds upon LiteRT with a segmented, mixed-precision model pipeline, trading a degree of raw efficiency for portability and reproducibility on commodity hardware.

## 3.3 Important Gaps

Our review of the literature on TinyML-based processing of sEMG signals revealed a notable gap: few existing implementations employ general-purpose inference frameworks such as LiteRT for this application. The majority of studies instead rely on specialised, kernel-level implementations, which deliver high efficiency but are tightly coupled to a particular architecture and therefore difficult to port to other platforms.

A further distinction concerns the design methodology. Most prior work adopts a hardware–model co-design approach, in which the model architecture and its parameters are developed jointly with the target hardware so as to maximise efficiency. In contrast, this thesis focuses on non-destructive deployment methods that operate independently of the underlying model design: the pipeline takes a pre-trained model as given and adapts it for embedded inference without altering its architecture or retraining its parameters. This separation of concerns favours portability and reusability, allowing a model developed independently of any hardware target to be deployed on resource-constrained devices.

A related limitation of prior work is that it typically targets specialised embedded processors that are not widely available commercially. To the best of our knowledge, this is the first study to conduct such experiments on Espressif-based systems. This is a notable gap, given that the ESP32 series has become a de facto standard for IoT devices, offering substantial computational capability relative to its low cost and broad commercial availability.

Finally, recurrent modules are seldom used in TinyML sEMG applications, owing to their comparatively high computational cost on resource-constrained hardware. We are not aware of any comprehensive study employing LSTMs for TinyML-based sEMG inference, still less one that implements an LSTM on a LiteRT-based system. Addressing these gaps, the use of a portable

inference framework, a non-destructive and model-independent deployment methodology, deployment on commodity Espressif hardware, and the inclusion of a recurrent module, constitutes the central motivation for this work.

# 4 Materials and Methods

## 4.1 Dataset Description and Preprocessing

### 4.1.1 NinaPro DB2 Dataset

The Non-Invasive Adaptive Hand Prosthetics (NinaPro) database is the largest publicly available multi-modal benchmark assembled to support machine-learning research on human, robotic, and myoelectric hand prostheses. The database comprises ten datasets totalling more than 180 data acquisitions from both intact subjects and transradial amputees, spanning electromyographic, kinematic, inertial, clinical, neurocognitive, and eye–hand coordination modalities. Each acquisition is obtained by simultaneously recording surface electromyography (sEMG) from the forearm together with the kinematics of the hand and wrist while the subject performs a predefined set of movements and postures [28, 29].

For the pre-processing, [8] employed the second NinaPro database (DB2), which contains recordings from 40 intact subjects performing 49 distinct gestures, each repeated six times. The sEMG signals, sampled at 2 KHz, were acquired using twelve Delsys Trigno wireless electrodes, and the corresponding hand kinematics were captured using a CyberGlove II data glove; the dataset additionally provides inertial and force modalities. For each recording, DB2 supplies twelve sEMG channels, thirty-six accelerometer channels (the three-axis accelerometers integrated into the twelve electrodes), and twenty-two glove channels containing the uncalibrated outputs of the CyberGlove sensors, which are approximately proportional to the joint angles of the hand and wrist [28, 29].

#### Channel and Gesture Selection

[8] restricted the data to a subset of channels and gestures. Of the twelve available sEMG channels, we retain only the first eight, which correspond to electrodes equally spaced around the forearm at the level of the radio-humeral joint. On the output side, the regression target is reduced to two glove channels, selected to capture the degrees of freedom most relevant to the gestures under study: one channel associated with finger abduction and fist closure, and one associated with wrist extension and flexion. For each gesture, the glove channel that is not actuated by that movement is set to zero, so that the model is trained to predict only the degree of freedom genuinely exercised by the gesture.

From the full set of 49 gestures, five were selected: abduction of all fingers, fingers flexed together into a fist, wrist flexion, wrist extension, and wrist extension with a closed hand; the remaining gestures are discarded. This yields a focused regression task over a small, well-characterized set of movements suited to deployment on a resource-constrained device.

**Preprocessing**

Each trial is band-pass filtered using a fourth-order Butterworth filter with a passband of 5–450 Hz, attenuating frequency components outside this range. Every trial is epoched to a fixed length of five seconds, corresponding to 10,000 samples at the DB2 sampling rate. The epoched recordings are then segmented into non-overlapping windows; that is, the windowing hop is set equal to the window length, so that consecutive windows tile each trial without overlap. The window length itself is treated as an experimental parameter, allowing its effect on both predictive accuracy and on-device latency to be studied systematically.

## 4.2 Deep Learning Model Architecture

The model architecture used in this work was developed by [8] at the Institute for Signal Processing and System Theory, University of Stuttgart, trained on the NinaPro DB2 dataset. It is a fully convolutional–recurrent regression network that maps a windowed, multi-channel sEMG input of shape $(B, C, T)$—where $B$ is the batch size, $C$ the number of input channels, and $T$ the window length—to a continuous estimate $\hat{y}$. For the present study, the input comprises $C = 8$ EMG channels. The network is organized into four functional stages: a convolutional feature extractor (input block), a convolutional downsampling block, a recurrent block, and a regression head, as illustrated in Figure 4.1.

**Input block (feature extraction).** The input block performs feature learning along the temporal and spatial (inter-channel) axes in sequence. The windowed signal is first expanded to $(B, 1, C, T)$ and passed through a temporal two-dimensional convolution with 48 output channels, a kernel of size $(1, 51)$ and same padding, followed by batch normalization. A spatial convolution with a kernel of size $(C, 1)$ and channel-wise grouping then mixes information across the input channels, after which the tensor is squeezed back to $(B, 48, T)$ and subjected to a further batch normalization, an ELU activation, and dropout with a rate of 0.5. The output of the input block is concatenated with the original input along the channel dimension, yielding a feature representation of shape $(B, 48 + C, T)$. This concatenation acts as a residual-style skip connection that preserves the raw input alongside the learned features.

**Downsampling block.** The concatenated feature map is downsampled along the temporal axis by a strided, grouped one-dimensional convolution with $48 + C$ output channels, a kernel of size 9, same padding, and a stride equal to the downsampling factor of 2. This is followed by batch normalization, an ELU activation, and dropout with a rate of 0.5, producing a tensor of shape $(B, 48 + C, T/2)$ in which the temporal resolution has been halved.

**Recurrent block.** Temporal dependencies are modelled by a two-layer unidirectional LSTM with a hidden size of $48 + C = 56$, which preserves the channel dimension of its input and outputs a sequence of shape $(B, 48 + C, T/2)$.

**Regression head.** The final stage is a fully connected regressor that maps the LSTM output to the target. It consists of linear layers with a geometric falloff that map $48 + C$ channels to the final regression output of 2 channels with tanh activations between the layers, not after output.

### Model Configuration

The architecture is highly configurable. The configuration evaluated in this work uses a dropout rate of 0.5 throughout, a downsampling stride of 2, and a two-layer LSTM with a hidden size of 56. The input block employs an undilated temporal convolution with a wide kernel: a kernel size of 51, 48 output channels, and a dilation of 1, with the regression head comprising two layers. The configuration is summarized in Table 4.1.

Table 4.1: Hyperparameter configuration of the evaluated model.

| Block | Hyperparameter | Value |
|---|---|---|
| Input | Output channels | 48 |
| | Kernel size | 51 |
| | Dilation | 1 |
| | Dropout | 0.5 |
| Downsample | Kernel size | 9 |
| | Stride | 2 |
| | Dropout | 0.5 |
| LSTM | Layers | 2 |
| | Hidden size | 56 |
| Regressor | Layers | 2 |

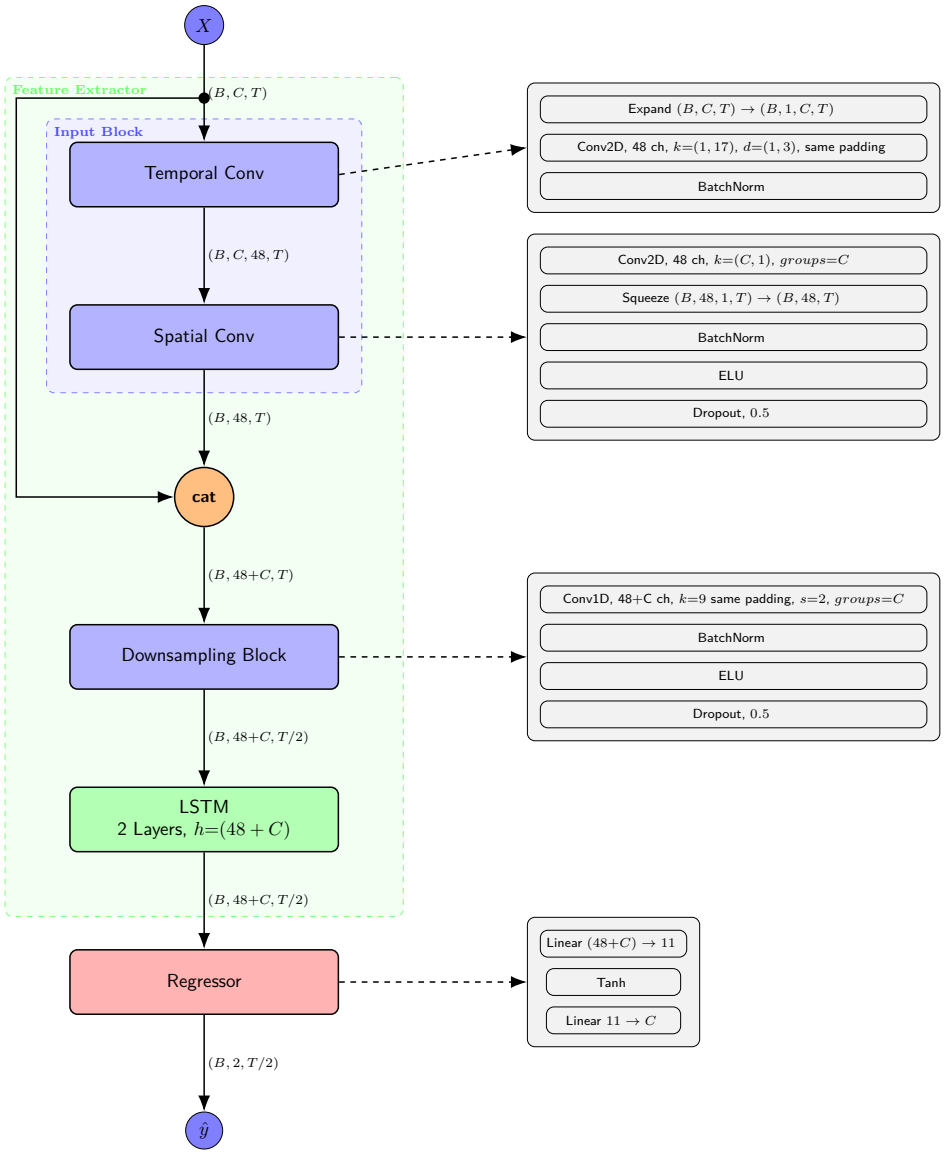



Figure 4.1: Deep Learning Architecture used for EMG regression by Koellner and Yang.

## 4.3 Hardware

### 4.3.1 ESP32-S3 Specifications

All experiments in this work were conducted on an ESP32-S3-based development board (ESP32-S3 UNO) equipped with 16 MB of external flash and 8 MB of external PSRAM (pseudo-static RAM). The ESP32-S3 was selected because it is one of the few low-cost microcontrollers whose architecture incorporates dedicated hardware acceleration for the integer arithmetic that dominates quantized neural-network inference. As noted in the motivation, an Espressif MCU was likewise chosen in response to the notable scarcity of TinyML sEMG studies addressing deployment on such platforms. This subsection summarizes the characteristics of the platform that are most relevant to on-device machine learning and to the single-instruction-multiple-data (SIMD) execution exploited by the ESP-NN backend. All figures are taken from

the official *ESP32-S3 Technical Reference Manual* [30].

### Processor and Memory Architecture

The ESP32-S3 is a dual-core system built around two Harvard-architecture Tensilica Xtensa® LX7 CPUs, each capable of operating at a clock frequency of up to 240 MHz. On-chip memory comprises 384 KB of internal ROM and 512 KB of internal SRAM, the latter accessible by the CPU typically within a single clock cycle. This relatively small fast-memory budget is a defining constraint of the platform: the complete set of model parameters, the interpreter's tensor arena, and all runtime buffers must either fit within the internal SRAM or be placed in the slower external memory.

To accommodate larger models, the ESP32-S3 supports external flash and external RAM through its SPI interfaces (including the Octal SPI and OPI modes used by high-bandwidth PSRAM). The CPU does not access external memory directly; instead, the external address space is mapped into the CPU address space through a cache. This indirection is significant for inference latency: data resident in the 8 MB PSRAM of our board is reached through the cache rather than at SRAM speed, so any working set (notably a large activation arena) that spills out of internal SRAM into PSRAM incurs additional access latency on cache misses.

### Processor Instruction Extensions (PIE) and SIMD

The feature most relevant to this work is the ESP32-S3's Processor Instruction Extensions (PIE), a vector instruction set defined through the Tensilica Instruction Extension (TIE) language and added specifically to accelerate AI and digital-signal-processing workloads. The PIE follows the SIMD paradigm, applying a single instruction across multiple data elements packed into wide vector registers, and supports 8-bit, 16-bit, and 32-bit integer vector operations.

## 4.4 Hardware Configuration

Espressif boards are highly configurable, which gives them a significant edge over their competitors. It is important to document the configuration used in this implementation in order to fully characterise the hardware environment in which the benchmarks were obtained. All experiments were conducted under the Espressif IoT Development Framework (ESP-IDF) version 6.0.0 with the GCC toolchain, targeting the ESP32-S3.

The CPU was clocked at its maximum frequency of 240 MHz, establishing the baseline against which all latency measurements should be interpreted. Crucially for this work, the ESP-NN backend was enabled in its optimised configuration (`CONFIG_NN_OPTIMIZED`), so that the assembly-optimised `int8` kernels discussed in Section 4.6.1 were used rather than the generic ANSI-C reference implementations; this setting underpins the `int8`-versus-floating-point latency behaviour reported in the results.

The memory configuration is likewise central to the measurements, given the SRAM-versus-PSRAM latency effects examined throughout. The board was configured with 8 MB of external Octal PSRAM operating at 80 MHz, integrated into the heap via the standard allocator (`CONFIG_SPIRAM_USE_MALLOC`). The allocation thresholds that govern whether a given buffer

is placed in fast internal SRAM or external PSRAM were left at their defaults, reserving 32 KB of internal memory and routing allocations below 16 KB to internal SRAM. The instruction and data caches, through which all external-memory accesses are mediated, were configured at 16 KB and 32 KB respectively, both with a 32-byte line size and eight-way associativity.

Finally, the device was fitted with 16 MB of external flash operating at 80 MHz in DIO mode, and a custom partition table was used to reserve a dedicated partition into which the packed model binaries were flashed, as described in Section 4.6.1.

## 4.5 Quantization Scheme Used

In this work, we opted for a PTQ quantization scheme. In order to comply with ESP-NN backend functions, we use full int8 quantization across the entire model with int32 accumulators and int32 for biases as presented here [23]. Since ESP-NN uses the same quantization schemes as TFlite runtime, we are using per tensor asymmetric quantization for inputs and activations while using symmetric per channels quantization for both weights and biases. Similar 8-bit PTQ schemes, with per channel quantization for weights and per layer quantization of activations, have shown accuracies within 2 percent of floating point networks, particularly for CNN architectures [22].

## 4.6 Software and Tools

### 4.6.1 ESP-NN Library

The ESP-NN library contains optimized NN (Neural Network) functions for various Espressif chips. They are the backend used by TensorFlow Lite Micro on esp32 chips. It supports ESP32, ESP32-S3, ESP32-P4 and ESP32-C3 chips. While it provides generic optimizations for most chips, it provide assembly optimized versions for ML dedicated chips like ESP32-S3.

The optimized ESP-NN libraries on ESP32-S3 chip are only dedicated for Int8 quantized operations. For matrix multiplications, the core arithmetic operations behind fully connected and LSTM layers, 2 functions are available for use. `esp_nn_fully_connected` which is used for per tensor weight quantization and the `esp_nn_fully_connected_per_ch` variant for per channel quantized weights.

Similar to [23], ESP-NN uses int32 accumulators for accumulating the intermediate products in matmul operations. This follows directly from the integer-arithmetic-only formulation of [23], where each real value $r$ is represented through an affine mapping $r = S\,(q - Z)$ from its quantized integer $q$, with scale $S$ and zero-point $Z$. Under this mapping, a quantized matrix product reduces to:

$$q_3^{(i,k)} = Z_3 + M \sum_{j=1}^{N} \left(q_1^{(i,j)} - Z_1\right)\left(q_2^{(j,k)} - Z_2\right), \tag{4.1}$$

where the summation is performed entirely in integers and held in a wide (int32) accumulator, exactly as ESP-NN does. The only non-integer term is the multiplier $M = S_1 S_2 / S_3$, which depends only on the fixed tensor scales and is therefore constant at inference. Since $M \in (0, 1)$, it is expressed in the fixed-point form $M = 2^{-n} M_0$ with $M_0 \in [0.5, 1)$. This is precisely how

the rescaling is discretized into the kernel's two integer parameters: $M_0$ becomes the fixed-point multiplier `out_mult`, and the factor $2^{-n}$ becomes a rounding right-shift by `out_shift`. Requantization thus reduces to one integer multiply followed by a shift, keeping the matmul pipeline entirely integer-only.

### 4.6.2 LiteRT (TensorFlow Lite) Framework

LiteRT is the successor to the widely adopted TensorFlow Lite pipeline. Developed by Google, TensorFlow Lite was designed to streamline TinyML workflows by providing a means of converting pre-trained TensorFlow models into a flatbuffer representation (the `.tflite` format). This flatbuffer is subsequently read by the TensorFlow Lite Micro interpreter on the target device to perform inference. A principal strength of the framework is that it delegates execution to the optimized neural-network kernels native to the host microcontroller—such as ESP-NN for Espressif devices and CMSIS-NN for ARM-based devices—thereby achieving near-native performance without requiring the developer to implement the model from scratch using low-level kernels.

The underlying development pipeline of TensorFlow Lite Micro (TFLM) comprises two distinct stages, namely an offline conversion stage performed on a host machine and an on-device inference stage executed on the target microcontroller [21]. In the offline stage, a model trained within the TensorFlow environment is processed by the TensorFlow Lite converter, which applies optimisations such as quantisation and the folding of constant expressions (e.g. batch normalisation) before serialising the result into a portable FlatBuffer file (`.tflite`). The FlatBuffer format is well suited to embedded targets, as its memory-mapped representation requires no unpacking at run time and can be readily embedded into the application binary as a C source array, obviating the need for a file system on devices that lack one [21].

In the on-device stage, rather than generating native code from the model, TFLM adopts an interpreter-based design in which a single, static body of execution code reads the FlatBuffer at run time to determine which operators to invoke and from where to draw their parameters [21]. This approach favours portability and field-updatability over the marginal performance gains of code generation, and incurs negligible overhead in practice, since the long-running neural-network kernels dominate execution time and amortise the cost of interpretation [21]. At initialisation, the application supplies the interpreter with the model, an operator resolver (`OpResolver`) that maps the operators present in the model to their implementations whilst minimising binary size, and a contiguous, fixed-size memory region termed the *arena*. The interpreter plans and allocates all required buffers from this arena during the initialisation phase and performs no further dynamic allocation during inference, thereby avoiding the heap fragmentation that would otherwise jeopardise long-running embedded applications [21]. It is precisely at the operator level of this interpreter that the aforementioned vendor-optimised kernel libraries are substituted for the portable reference implementations, allowing hardware-specific acceleration without modification to the surrounding framework [21].

The principal limitation of TensorFlow Lite was its tight coupling to the TensorFlow ecosystem, which required PyTorch models to be translated into TensorFlow before conversion. This translation was not straightforward: it entailed first exporting the PyTorch model to the ONNX interchange format and then converting the ONNX model to TensorFlow. In our experience, this multi-stage pipeline incurred substantial conversion overhead and, in many cases, failed to complete owing to discrepancies between the PyTorch and TensorFlow model representations—

for example, differences in tensor memory layout (NCHW versus NHWC) and in the expected index of the batch dimension when exporting LSTM layers.

Released in 2024, LiteRT removes this dependency on the TensorFlow ecosystem, repositioning the framework as a cross-framework solution that no longer requires intermediate conversion formats, while retaining the `.tflite` file format for backward compatibility. LiteRT additionally exposes a range of configurable quantization schemes, including Float16, hybrid, and full Int8 quantization.

## 4.7 Implementation Details and Experimental Setup

### 4.7.1 Problems with Direct Conversion

Our initial approach was to convert the entire network into a single flatbuffer and deploy it as one monolithic graph. This proved unworkable for several distinct reasons, which together motivated the modular strategy described in the following section.

The first and most serious issue was the recurrent stage. Although TensorFlow Lite Micro on the ESP32-S3 nominally exposes a `UnidirectionalLSTM` operator through its `MutableOpResolver`, exporting the LSTM via this operator did not function correctly in our experiments. We therefore implemented the LSTM manually as an unrolled cell and copied the trained parameters from the original model into it.

The second issue arose from a large concatenation at the output of the recurrent stage, which had to assemble the entire downsampled window of 250 timesteps. TensorFlow Lite Micro restricts each concatenation operator to at most ten inputs, forcing this single operation to be expressed as a tree of concatenation blocks, each combining at most ten inputs. This restructuring inflated both the operation count and the model size, which reached 2.8 MB for the full network, far beyond a comfortable budget for the target MCU, and produced a prohibitive per-window latency of approximately 5,000 ms.

Quantization compounded these difficulties. Exporting the convolutional feature extractor as a single `int8` block proved inadequate for three reasons. The ELU activations, which lack a native ESP-NN kernel, were decomposed by the converter into primitive `Exp`, `Sub`, and comparison operations, degrading performance. The one-dimensional convolutions were not quantized by the default PT2E quantizer. Finally, the `BatchNorm1d` layers were realized as separate operations that executed in floating point even within an otherwise `int8` graph. Each of these is addressed by a dedicated graph-level transformation below.

### 4.7.2 Modular Architecture

To overcome the limitations of the monolithic export, we adopted a modular deployment strategy in which the network is partitioned at carefully chosen points rather than exported as a single long graph. Partitioning serves three purposes: it eliminates redundant operations, it permits the quantization scheme and numerical precision to be selected independently for each part, and it keeps the working set of any one stage small. The partition points were chosen experimentally with the joint objectives of minimizing the mean squared error and reducing the memory footprint, yielding three split points that divide the model into four stages.

The partitioning was performed non-destructively: each stage was reconstructed as a standalone model from the trained network, with the corresponding parameters copied into it, so that neither the architecture nor the trained weights were altered. The four resulting stages are the two convolutional feature-extraction models (the temporal and spatial convolutions of the input block forming the first stage, and the downsampling convolution the second), the two-layer LSTM cell as the third stage, and the regression head as the fourth.

The export of the recurrent stage is central to this strategy. Rather than unrolling the LSTM across the entire window, which produced the prohibitive footprint and latency noted above, the LSTM is exported as a single block spanning only a small, fixed number of timesteps and is then invoked repeatedly within an on-device loop, together with the regression head, to process the full sequence. Because a single short block resides in memory and is reused across iterations, rather than the full set of unrolled timesteps being materialized as distinct cells, this windowed export sharply reduces the working memory of the recurrent stage. The number of timesteps per block, the *LSTM step size*, thus becomes a deployment parameter trading recurrent loop overhead against working memory, the effect of which is characterised in the results.

Taken together, these measures reduced the footprint of the complete model from 2.8 MB to approximately 260 KB. Equally importantly, the decomposition enabled mixed-precision deployment, allowing accurate floating-point recurrent stages to operate alongside optimized `int8` convolutional stages.

### 4.7.3 Graph-Level Transformations for Quantization

Three non-destructive transformations of the model graph were required to obtain correct and efficient quantized stages, each addressing one of the quantization problems identified above.

First, the batch-normalization layers were folded into the weights and biases of the preceding convolutions and removed from the graph. The converter was unable to fuse the `BatchNorm1d` layers itself, realizing each instead as a separate multiply–add pair placed inside a floating-point quantize/dequantize island; folding them removed both the redundant computation and the associated precision conversions.

Second, the one-dimensional convolutions were reformulated as two-dimensional convolutions with a singleton spatial dimension, since the LiteRT and TensorFlow Lite Micro toolchain does not quantize `Conv1D` operations by default.

Third, the first ELU activation block was removed from the graph and reimplemented manually as an `int8` look-up table, avoiding the decomposed floating-point island produced by the converter; the second ELU block was retained in floating point. The concatenation of the residual point also had to be reimplemented to account for manually implementing the second ELU.

### 4.7.4 Experimental Workflow

The experimental workflow proceeds as follows. A configuration-driven system accepts a list of model configurations, specifying parameters such as the window length and the LSTM step size, and batch-converts each to the LiteRT format, exporting eight models per configuration: the four pipeline stages in floating point and the same four stages `int8`-quantized. The exported models are packed into a single contiguous binary and flashed to a dedicated partition of the microcontroller's flash memory. The complete input trial is exported alongside them, together

with the reference output of the original PyTorch model, against which the normalized mean squared error is computed. On the ESP32-S3, a continuous loop iterates over all configurations to generate the final results.

It is crucial to emphasise that the NMSE, and all accuracy metrics reported in this work, were computed relative to the outputs of the original floating-point PyTorch model rather than against the NinaPro DB2 ground-truth labels. This choice follows directly from the scope of the thesis established earlier: the present work is concerned solely with the faithful deployment and implementation of a pre-trained model on resource-constrained hardware, not with its predictive accuracy on the underlying task. The reference model's outputs therefore constitute the appropriate ground truth, as the objective is to quantify the fidelity loss introduced by the deployment pipeline itself, in isolation from any error intrinsic to the model.

### 4.7.5 Evaluation Metrics

The performance of the LiteRT models on the ESP32-S3 was assessed along three dimensions: latency, fidelity, and memory footprint. Latency was measured in microseconds. The inference times of the network stages were obtained using the MicroProfiler tool provided by TensorFlow Lite Micro, with the device configured such that one tick corresponds to one microsecond. The manually implemented ELU stages lie outside the TensorFlow Lite Micro interpreter and were therefore timed separately using the `esp_timer` high-resolution timer, which likewise reports in microseconds.

Model fidelity was quantified using the normalised mean squared error (NMSE), defined as the squared error norm normalised by the variance of the reference signal. This measure is dimensionless and therefore comparable across configurations. For the ML-operation benchmarks, the unnormalised mean squared error (MSE) was used instead, as the inputs were initialised with random weights drawn from a standard normal distribution (zero mean, unit variance), for which normalisation is unnecessary. All memory measurements are reported in kilobytes.

Moreover, window sizes are reported in the results section in terms of samples. The corresponding temporal span is obtained by dividing the sample count by the sampling rate:

$$T_{\text{win}} = \frac{N}{f_s}, \tag{4.2}$$

where $T_{\text{win}}$ is the window span in seconds, $N$ the window size in samples, and $f_s$ the sampling rate in hertz. For the NinaPro DB2 recordings used here, sampled at $f_s = 2$ kHz, a window of $N = 500$ samples therefore corresponds to a span of 250 ms.

# 5 Results

## 5.1 Comparison of ML Ops Performance on the ESP32-S3 using Random Weights under different Quantization Schemes

In order to evaluate general Model performance on the esp32s3, we had to create our own small benchmark to evaluate the behavior of different data layers and how they react to different parameter changes and to int8 quantization used. Randomly chosen weights and inputs follow the standardized normal distribution.

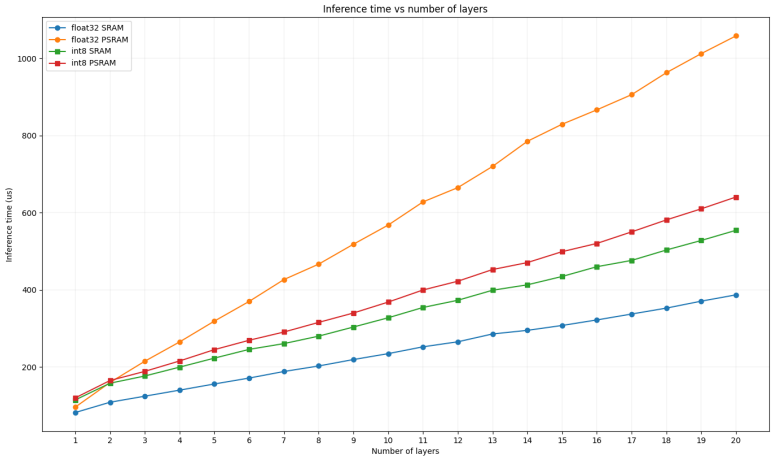



Figure 5.1: Figure shows latency of dense layers against the amount of layers used in a geometric decay.

The above graph shows how the number of layers impacts the final latency for dense layers, where a geometric falloff determines the number of units at each stage, decaying from 50 down to 2. Across all four configurations latency grows approximately linearly with the number of layers. The effect of int8 quantization, however, depends strongly on the memory in which the model resides. On PSRAM, int8 quantization reduces latency relative to float32 (e.g. $\sim$ 640 $\mu s$ vs. $\sim$ 1060 $\mu s$ at 20 layers), as the four-fold reduction in weight size lowers traffic over the slow external bus on this memory-bound workload. On SRAM the trend reverses: int8 is consistently slower than float32 (e.g. $\sim$ 555 $\mu s$ vs. $\sim$ 386 $\mu s$ at 20 layers), because the workload becomes compute-bound and the ESP32-S3's hardware floating-point unit executes each float multiply-accumulate in a single instruction, whereas the int8 path incurs additional per-channel requantization overhead. The fastest configuration overall is float32 on SRAM, and the slowest is float32 on PSRAM.

Table 5.1: Conv2D layers performance on the ESP32-S3.

| | | | PSRAM | | | | No PSRAM | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | Float32 | | Int8 | | Float32 | | Int8 | |
| | Input | Architecture | Inf ($\mu$s) | MSE | Inf ($\mu$s) | MSE | Inf ($\mu$s) | MSE | Inf ($\mu$s) | MSE |
| Layers | (1, 2, 50) | Conv(in=1, out=10, L=1, k=(1, 5)) | 1813 | 0 | 1198 | 0.0003 | 1574 | 0 | 1191 | 0.0003 |
| | (1, 2, 50) | Conv(in=1, out=10, L=2, k=(1, 5)) | 2933 | 0 | 1768 | 0.0005 | 2908 | 0 | 1755 | 0.0005 |
| | (1, 2, 50) | Conv(in=1, out=10, L=3, k=(1, 5)) | 2807 | 0 | 1701 | 0.0036 | 2785 | 0 | 1680 | 0.0036 |
| | (1, 2, 50) | Conv(in=1, out=10, L=4, k=(1, 5)) | 2464 | 0 | 1556 | 0.0022 | 2453 | 0 | 1548 | 0.0022 |
| Input Size | (1, 10, 10) | Conv(in=1, out=10, L=1, k=(2, 2)) | 1636 | 0 | 1133 | 0.0002 | 1472 | 0 | 1125 | 0.0002 |
| | (1, 20, 20) | Conv(in=1, out=10, L=1, k=(2, 2)) | 7209 | 0 | 3817 | 0.0004 | 5597 | 0 | 3477 | 0.0004 |
| | (1, 30, 30) | Conv(in=1, out=10, L=1, k=(2, 2)) | 23227 | 0 | 8489 | 0.0006 | 12642 | 0 | 7500 | 0.0006 |
| | (1, 40, 40) | Conv(in=1, out=10, L=1, k=(2, 2)) | 50534 | 0 | 15464 | 0.0005 | 22606 | 0 | 13179 | 0.0005 |
| Kernel Size | (1, 1, 100) | Conv(in=1, out=10, L=1, k=(1, 5)) | 1832 | 0 | 1217 | 0.0003 | 1628 | 0 | 1219 | 0.0003 |
| | (1, 1, 100) | Conv(in=1, out=10, L=1, k=(1, 10)) | 2338 | 0 | 1308 | 0.0002 | 2260 | 0 | 1305 | 0.0002 |
| | (1, 1, 100) | Conv(in=1, out=10, L=1, k=(1, 20)) | 3386 | 0 | 1458 | 0.0002 | 3297 | 0 | 1436 | 0.0002 |
| | (1, 1, 100) | Conv(in=1, out=10, L=1, k=(1, 30)) | 4118 | 0 | 1526 | 0.0002 | 4014 | 0 | 1514 | 0.0002 |
| Output Ch | (1, 1, 100) | Conv(in=1, out=1, L=1, k=(1, 5)) | 389 | 0 | 448 | 0.0002 | 386 | 0 | 445 | 0.0002 |
| | (1, 1, 100) | Conv(in=1, out=20, L=1, k=(1, 5)) | 3439 | 0 | 1433 | 0.0003 | 2943 | 0 | 1353 | 0.0003 |
| | (1, 1, 100) | Conv(in=1, out=30, L=1, k=(1, 5)) | 5077 | 0 | 2694 | 0.0003 | 4253 | 0 | 2469 | 0.0003 |

Table 5.2: Conv2D layers performance on the ESP32-S3.

| | | | PSRAM | | | | No PSRAM | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | Float32 | | Int8 | | Float32 | | Int8 | |
| | Input | Architecture | Size (KB) | Arena (KB) | Size (KB) | Arena (KB) | Size (KB) | Arena (KB) | Size (KB) | Arena (KB) |
| Layers | (1, 2, 50) | Conv(in=1, out=10, L=1, k=(1, 5)) | 2.3 | 7.9 | 2.6 | 2.6 | 2.3 | 7.9 | 2.6 | 2.6 |
| | (1, 2, 50) | Conv(in=1, out=10, L=2, k=(1, 5)) | 3.6 | 4.0 | 3.8 | 3.2 | 3.6 | 4.0 | 3.8 | 3.2 |
| | (1, 2, 50) | Conv(in=1, out=10, L=3, k=(1, 5)) | 4.9 | 2.9 | 4.8 | 3.4 | 4.9 | 2.9 | 4.8 | 3.4 |
| | (1, 2, 50) | Conv(in=1, out=10, L=4, k=(1, 5)) | 6.0 | 3.0 | 5.8 | 3.5 | 6.0 | 3.0 | 5.8 | 3.5 |
| Input Size | (1, 10, 10) | Conv(in=1, out=10, L=1, k=(2, 2)) | 2.3 | 7.1 | 2.6 | 2.4 | 2.3 | 7.1 | 2.6 | 2.4 |
| | (1, 20, 20) | Conv(in=1, out=10, L=1, k=(2, 2)) | 2.3 | 29.0 | 2.6 | 7.8 | 2.3 | 28.9 | 2.6 | 7.8 |
| | (1, 30, 30) | Conv(in=1, out=10, L=1, k=(2, 2)) | 2.3 | 66.5 | 2.6 | 17.2 | 2.3 | 66.4 | 2.6 | 17.2 |
| | (1, 40, 40) | Conv(in=1, out=10, L=1, k=(2, 2)) | 2.3 | 119.6 | 2.6 | 30.5 | 2.3 | 119.6 | 2.6 | 30.5 |
| Kernel Size | (1, 1, 100) | Conv(in=1, out=10, L=1, k=(1, 5)) | 2.3 | 8.2 | 2.6 | 2.6 | 2.3 | 8.2 | 2.6 | 2.6 |
| | (1, 1, 100) | Conv(in=1, out=10, L=1, k=(1, 10)) | 2.5 | 7.9 | 2.6 | 2.6 | 2.5 | 7.9 | 2.6 | 2.6 |
| | (1, 1, 100) | Conv(in=1, out=10, L=1, k=(1, 20)) | 2.9 | 7.1 | 2.7 | 2.4 | 2.9 | 7.1 | 2.7 | 2.4 |
| | (1, 1, 100) | Conv(in=1, out=10, L=1, k=(1, 30)) | 3.3 | 6.3 | 2.8 | 2.2 | 3.3 | 6.3 | 2.8 | 2.2 |
| Output Ch | (1, 1, 100) | Conv(in=1, out=1, L=1, k=(1, 5)) | 1.7 | 1.4 | 1.9 | 0.9 | 1.7 | 1.4 | 1.9 | 0.9 |
| | (1, 1, 100) | Conv(in=1, out=20, L=1, k=(1, 5)) | 2.5 | 15.8 | 2.9 | 4.6 | 2.5 | 15.8 | 2.9 | 4.6 |
| | (1, 1, 100) | Conv(in=1, out=30, L=1, k=(1, 5)) | 2.8 | 23.4 | 3.2 | 6.6 | 2.8 | 23.4 | 3.2 | 6.5 |

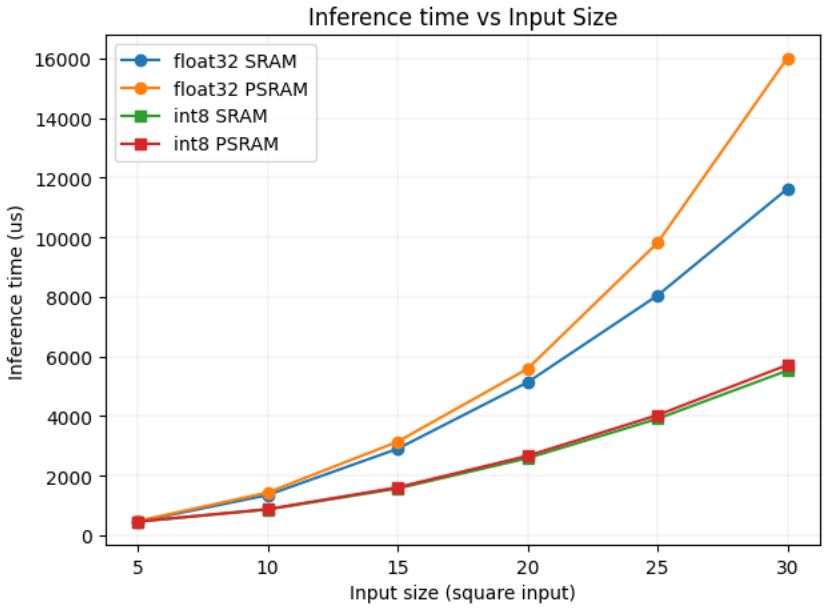



Figure 5.2: Figure shows latency of Conv2D layers against Input Size.

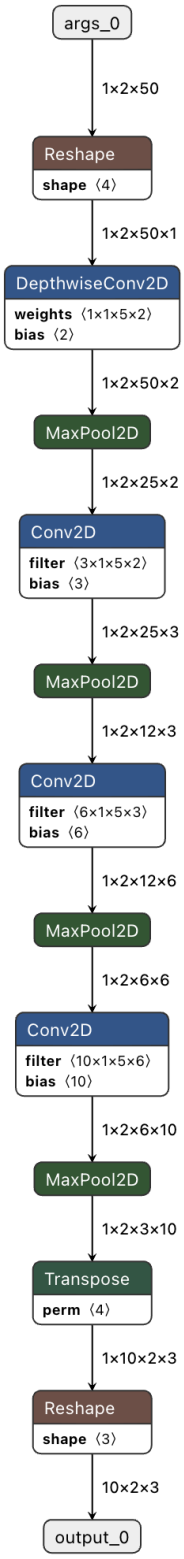



Figure 5.3: Figure shows Conv2D graph with 4 convolutional layers.

Across all Conv2D configurations, inference latency on internal SRAM is consistently lower than on PSRAM; for the $(1, 40, 40)$ input the float32 latency falls from 50534 $\mu s$ to 22606 $\mu s$ when the model is executed from SRAM. Int8 quantization yields a further substantial reduction, lowering the same configuration from 50534 $\mu s$ (float32, PSRAM) to 15464 $\mu s$ (int8, PSRAM), a

3.27× speedup, at a negligible accuracy cost (MSE ≤ 0.0006 throughout with data variation with random weights using normal distribution). The benefit of quantization is workload-dependent: for the smallest layer (a single output channel) int8 is marginally slower than float32 (448 $\mu s$ vs. 389 $\mu s$), whereas its relative advantage widens as the layer grows, reaching the 3.27× factor noted above.

Latency increases monotonically with input size, kernel size, and the number of output channels, as expected from the corresponding growth in the convolution workload. The layer-count sweep is the exception: because the architecture employs "same" convolutions with a MaxPool2D stage after each layer and assigns intermediate channel counts via geometric decay, each additional layer both halves the spatial extent of the feature map (Fig. 5.3, $50 \to 25 \to 12 \to 6$) and alters the distribution of intermediate channel widths. As a result, the per-layer cost falls rapidly with depth, and total latency is not monotonic in the number of layers; the four-layer model (1556 $\mu s$, int8 PSRAM) is faster than the three-layer model (1701 $\mu s$) because its intermediate channel allocation yields less work despite the additional stage.

The memory footprint (Table 5.2) exhibits a clear separation between model size and arena (working-memory) requirements. Model size is largely insensitive to quantization for these small networks; because the parameter counts are modest, the per-tensor quantization metadata roughly offsets the four-fold reduction in weight precision, so int8 models are in fact marginally larger than their float32 counterparts for the smallest configurations (e.g. 2.3 KB → 2.6 KB for the single-layer $k$=(1, 5) model), and only become smaller once the weight tensors dominate (e.g. 3.3 KB → 2.8 KB at $k$=(1, 30)). The arena, by contrast, is dominated by activation tensors and is where quantization delivers its principal benefit: for the (1, 40, 40) input the arena falls from 119.6 KB (float32) to 30.5 KB (int8), a 3.9× reduction. Arena size grows steeply with the activation volume, increasing with input size (7.1 → 119.6 KB for float32 as the input grows from 10×10 to 40×40) and with the number of output channels (1.4 → 23.4 KB from one to thirty channels), while remaining identical between the PSRAM and No-PSRAM configurations, confirming that memory placement affects latency but not footprint.

Table 5.3: LSTM layers performance on the ESP32-S3.

| | | | PSRAM | | | | No PSRAM | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | Float32 | | Int8 | | Float32 | | Int8 | |
| | Input | Architecture | Inf ($\mu$s) | MSE | Inf ($\mu$s) | MSE | Inf ($\mu$s) | MSE | Inf ($\mu$s) | MSE |
| Time-Steps | (1, 1, 25) | LSTM(25, 25, 1) | 1417 | 0 | 1523 | 0 | 896 | 0 | 1432 | 0 |
| | (5, 1, 25) | LSTM(25, 25, 1) | 3736 | 0 | 5731 | 0 | 2495 | 0 | 4894 | 0 |
| | (10, 1, 25) | LSTM(25, 25, 1) | 6513 | 0 | 11234 | 0 | 4409 | 0 | 9431 | 0 |
| | (15, 1, 25) | LSTM(25, 25, 1) | 9542 | 0 | 17538 | 0.0036 | 6460 | 0 | 14733 | 0.0036 |
| | (20, 1, 25) | LSTM(25, 25, 1) | 12346 | 0 | 23874 | 0.0030 | —— | —— | —— | —— |
| Hidden Size | (1, 1, 50) | LSTM(50, 50, 1) | 4111 | 0 | 2836 | 0 | 1455 | 0 | 2268 | 0 |
| | (1, 1, 100) | LSTM(100, 100, 1) | 11961 | 0 | 8874 | 0 | —— | —— | —— | —— |
| | (1, 1, 150) | LSTM(150, 150, 1) | 24933 | 0 | 17038 | 0 | —— | —— | —— | —— |
| | (1, 1, 200) | LSTM(200, 200, 1) | 42847 | 0 | 27723 | 0 | —— | —— | —— | —— |
| Layers | (1, 1, 25) | LSTM(25, 25, 2) | 3225 | 0 | 3312 | 0.0045 | 1595 | 0 | 2870 | 0.0045 |
| | (1, 1, 25) | LSTM(25, 25, 3) | 4353 | 0 | 4764 | 0.0029 | 2054 | 0 | 3917 | 0.0029 |
| | (1, 1, 25) | LSTM(25, 25, 4) | 5540 | 0 | 6261 | 0.0050 | 2519 | 0 | 5020 | 0.0050 |
| | (1, 1, 25) | LSTM(25, 25, 5) | 6677 | 0 | 7776 | 0.0014 | 2968 | 0 | 6170 | 0.0014 |
| | (1, 1, 25) | LSTM(25, 25, 6) | 7867 | 0 | 9303 | 0.0005 | —— | —— | —— | —— |

Table 5.4: LSTM layers performance on the ESP32-S3.

| | | | PSRAM | | | | No PSRAM | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | Float32 | | Int8 | | Float32 | | Int8 | |
| | Input | Architecture | Size (KB) | Arena (KB) | Size (KB) | Arena (KB) | Size (KB) | Arena (KB) | Size (KB) | Arena (KB) |
| Time-Steps | (1, 1, 25) | LSTM(25, 25, 1) | 25.8 | 3.2 | 16.0 | 5.5 | 25.8 | 3.2 | 16.0 | 5.5 |
| | (5, 1, 25) | LSTM(25, 25, 1) | 41.4 | 9.2 | 43.5 | 21.1 | 41.4 | 9.2 | 43.5 | 21.1 |
| | (10, 1, 25) | LSTM(25, 25, 1) | 60.9 | 16.5 | 77.6 | 41.3 | 60.9 | 16.5 | 77.6 | 41.3 |
| | (15, 1, 25) | LSTM(25, 25, 1) | 81.5 | 24.4 | 113.2 | 63.2 | 81.5 | 24.4 | 113.2 | 63.2 |
| | (20, 1, 25) | LSTM(25, 25, 1) | 101.0 | 31.6 | 147.5 | 85.9 | —— | —— | —— | —— |
| Hidden Size | (1, 1, 50) | LSTM(50, 50, 1) | 84.4 | 4.5 | 33.0 | 8.3 | 84.4 | 4.5 | 33.0 | 8.3 |
| | (1, 1, 100) | LSTM(100, 100, 1) | 318.7 | 7.0 | 96.2 | 13.8 | —— | —— | —— | —— |
| | (1, 1, 150) | LSTM(150, 150, 1) | 709.4 | 9.6 | 198.6 | 19.3 | —— | —— | —— | —— |
| | (1, 1, 200) | LSTM(200, 200, 1) | 1256.2 | 12.1 | 340.0 | 24.8 | —— | —— | —— | —— |
| Output Size | (1, 1, 25) | LSTM(25, 25, 2) | 54.3 | 6.3 | 35.7 | 10.7 | 54.3 | 6.3 | 35.7 | 10.7 |
| | (1, 1, 25) | LSTM(25, 25, 3) | 80.6 | 8.8 | 52.9 | 15.5 | 80.6 | 8.8 | 52.9 | 15.5 |
| | (1, 1, 25) | LSTM(25, 25, 4) | 106.9 | 11.1 | 70.1 | 20.5 | 106.9 | 11.1 | 70.1 | 20.5 |
| | (1, 1, 25) | LSTM(25, 25, 5) | 133.2 | 13.2 | 87.3 | 25.1 | 133.2 | 13.2 | 87.3 | 25.1 |
| | (1, 1, 25) | LSTM(25, 25, 6) | 159.4 | 15.4 | 104.5 | 30.1 | —— | —— | —— | —— |

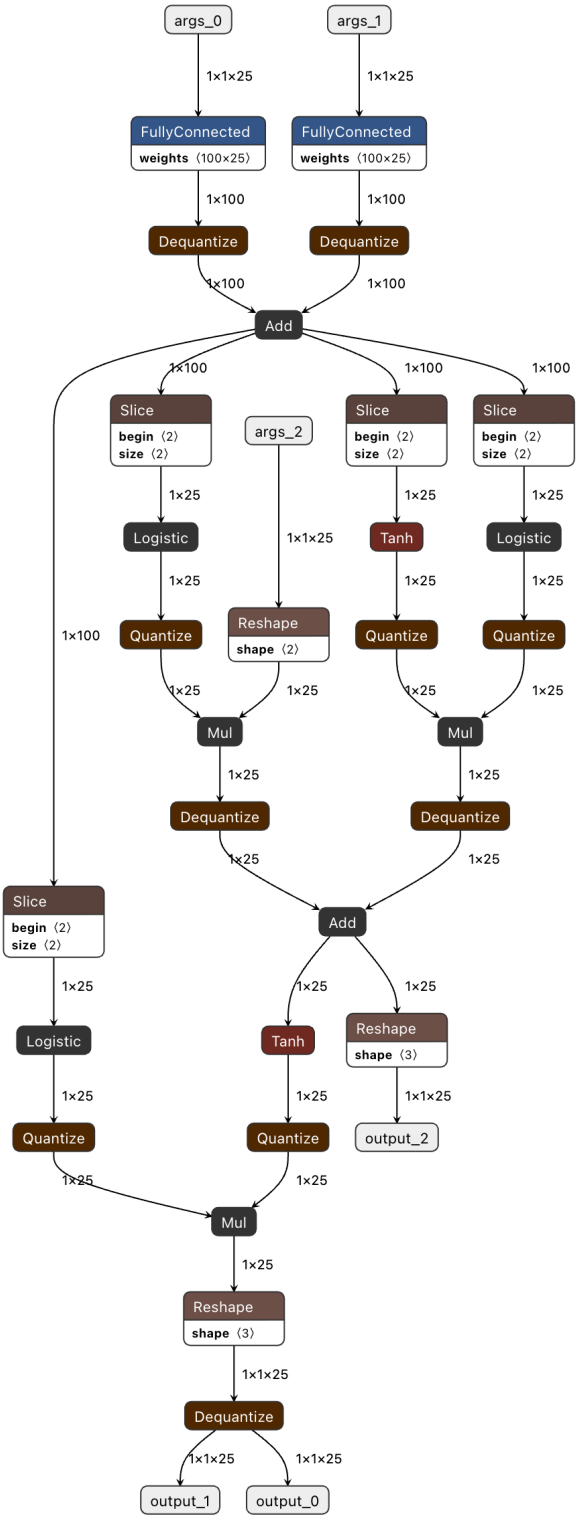



Figure 5.4: Figure shows the LSTM quanatized graph

Table 5.3 reports our benchmark of the manually implemented LSTM layers using the default LiteRT export and quantization pipeline. Inspecting the exported model layout reveals that the dominant source of int8 latency overhead is the set of intermittent `Quantize`/`Dequantize` stages (Fig. 5.4), which must traverse the entire activation tensors at each transition. The cost

of these stages scales with the number of such traversals per inference: in the time-step and layer-count sweeps, where these transitions recur, int8 inference is consistently slower than its float32 counterpart (e.g., 17538 $\mu s$ vs. 9542 $\mu s$ at 15 time steps on PSRAM). Notably, this trend reverses for large single-layer hidden sizes, where the matrix computation dominates the quantization overhead and int8 becomes faster than float32 (e.g., 27723 $\mu s$ vs. 42847 $\mu s$ for a hidden size of 200 on PSRAM).

A second observation concerns quantization quality. Single-layer, single-time-step configurations incur negligible error (MSE = 0), whereas accuracy degrades once additional time steps or stacked layers are introduced: the mean-squared error rises to 0.0036 at 15 time steps and reaches 0.0050 for the four-layer configuration. This indicates that the accumulation of quantization error across recurrent and stacked computations, rather than the LSTM operation in isolation, is the principal driver of the observed accuracy loss.

The memory footprint (Table 5.4) shows the opposite balance to the convolutional case. Because LSTM weight matrices scale quadratically with the hidden size, the model is parameter-dominated, and int8 quantization produces large reductions in model size: for a hidden size of 200 the model shrinks from 1256.2 KB (float32) to 340.0 KB (int8), a 3.7× saving. The arena, however, is consistently *larger* under int8 than under float32 (e.g. 12.1 KB → 24.8 KB at hidden size 200, and 31.6 KB → 85.9 KB at 20 time steps), reflecting the additional scratch buffers required by the intermittent `Quantize`/`Dequantize` stages discussed above. Arena requirements also grow with the number of time steps (3.2 → 31.6 KB for float32 from one to twenty steps), which, together with the inflated int8 arena, explains why several int8 configurations exceed the available internal SRAM and could only be executed from PSRAM (denoted by dashes in Table 5.3).

## 5.2 Computational Complexity (MAC Analysis)

Table 5.5: Multiply–accumulate operations per model stage across input window sizes.

| Cfg | T | N | Input | Down | LSTM step | Reg | LSTM total | Total |
|---|---|---|---|---|---|---|---|---|
| 0 | 50 | 1 | 998 400 (1.00) | 12 600 (0.01) | 50 512 (0.05) | 1 128 (0.00) | 1 262 800 (1.26) | 2 274 928 (2.27) |
| 1 | 50 | 5 | 998 400 (1.00) | 12 600 (0.01) | 252 560 (0.25) | 1 128 (0.00) | 1 262 800 (1.26) | 2 274 928 (2.27) |
| 2 | 100 | 1 | 1 996 800 (2.00) | 25 200 (0.03) | 50 512 (0.05) | 1 128 (0.00) | 2 525 600 (2.53) | 4 548 728 (4.55) |
| 3 | 100 | 5 | 1 996 800 (2.00) | 25 200 (0.03) | 252 560 (0.25) | 1 128 (0.00) | 2 525 600 (2.53) | 4 548 728 (4.55) |
| 4 | 150 | 1 | 2 995 200 (3.00) | 37 800 (0.04) | 50 512 (0.05) | 1 128 (0.00) | 3 788 400 (3.79) | 6 822 528 (6.82) |
| 5 | 150 | 5 | 2 995 200 (3.00) | 37 800 (0.04) | 252 560 (0.25) | 1 128 (0.00) | 3 788 400 (3.79) | 6 822 528 (6.82) |
| 6 | 200 | 1 | 3 993 600 (3.99) | 50 400 (0.05) | 50 512 (0.05) | 1 128 (0.00) | 5 051 200 (5.05) | 9 096 328 (9.10) |
| 7 | 200 | 5 | 3 993 600 (3.99) | 50 400 (0.05) | 252 560 (0.25) | 1 128 (0.00) | 5 051 200 (5.05) | 9 096 328 (9.10) |
| 8 | 250 | 1 | 4 992 000 (4.99) | 63 000 (0.06) | 50 512 (0.05) | 1 128 (0.00) | 6 314 000 (6.31) | 11 370 128 (11.37) |
| 9 | 250 | 5 | 4 992 000 (4.99) | 63 000 (0.06) | 252 560 (0.25) | 1 128 (0.00) | 6 314 000 (6.31) | 11 370 128 (11.37) |
| 10 | 300 | 1 | 5 990 400 (5.99) | 75 600 (0.08) | 50 512 (0.05) | 1 128 (0.00) | 7 576 800 (7.58) | 13 643 928 (13.64) |
| 11 | 300 | 5 | 5 990 400 (5.99) | 75 600 (0.08) | 252 560 (0.25) | 1 128 (0.00) | 7 576 800 (7.58) | 13 643 928 (13.64) |
| 12 | 350 | 1 | 6 988 800 (6.99) | 88 200 (0.09) | 50 512 (0.05) | 1 128 (0.00) | 8 839 600 (8.84) | 15 917 728 (15.92) |
| 13 | 350 | 5 | 6 988 800 (6.99) | 88 200 (0.09) | 252 560 (0.25) | 1 128 (0.00) | 8 839 600 (8.84) | 15 917 728 (15.92) |
| 14 | 400 | 1 | 7 987 200 (7.99) | 100 800 (0.10) | 50 512 (0.05) | 1 128 (0.00) | 10 102 400 (10.10) | 18 191 528 (18.19) |
| 15 | 400 | 5 | 7 987 200 (7.99) | 100 800 (0.10) | 252 560 (0.25) | 1 128 (0.00) | 10 102 400 (10.10) | 18 191 528 (18.19) |
| 16 | 450 | 1 | 8 985 600 (8.99) | 113 400 (0.11) | 50 512 (0.05) | 1 128 (0.00) | 11 365 200 (11.37) | 20 465 328 (20.47) |
| 17 | 450 | 5 | 8 985 600 (8.99) | 113 400 (0.11) | 252 560 (0.25) | 1 128 (0.00) | 11 365 200 (11.37) | 20 465 328 (20.47) |
| 18 | 500 | 1 | 9 984 000 (9.98) | 126 000 (0.13) | 50 512 (0.05) | 1 128 (0.00) | 12 628 000 (12.63) | 22 739 128 (22.74) |
| 19 | 500 | 5 | 9 984 000 (9.98) | 126 000 (0.13) | 252 560 (0.25) | 1 128 (0.00) | 12 628 000 (12.63) | 22 739 128 (22.74) |

Table 5.5 reports the number of multiply–accumulate operations (MACs) incurred by each stage of the model across the evaluated input window sizes. The MAC counts were derived from the convolutional, dense, and element-wise multiplication operations present in the model's `.tflite` buffers. MACs are a standard measure for assessing model complexity, as they provide a platform-independent estimate of computational intensity that is invariant to the particular hardware on which the model is ultimately deployed.

As shown in the table, the total MAC count grows approximately linearly with the input window length $T$, rising from 2.275 MMAC at $T = 50$ to 22.739 MMAC at $T = 500$. This trend is dominated by the input convolutional module and the per-timestep cost of the LSTM, both of which scale with the sequence length, whereas the regressor head contributes a negligible and constant 0.001 MMAC across all configurations. The MAC counts are identical for the `float32` and `int8` variants of each configuration, as expected: quantisation reduces the arithmetic precision and memory footprint of each operation but does not alter the number of operations performed.

It is worth emphasising that the model studied here is considerably more complex than the architectures typically employed in TinyML sEMG applications. For comparable input window sizes, Zanghieri et al. report a regression model requiring only 3.16 MMAC [27], while their earlier classification model is of a similar order [18]. Likewise, Faraone et al. report approximately 3.221 MOps for their convolutional–recurrent model [25], a figure of the same order of magnitude (noting that an operation count and a MAC count differ by roughly a factor of two, since one MAC comprises both a multiplication and an addition). Even the smallest configuration of our model (2.275 MMAC) is therefore comparable to these prior works, while the larger configurations exceed them by up to an order of magnitude, reflecting the additional cost of the two-layer LSTM and the richer feature-extraction front end.

## 5.3 Comparison of different Model Configurations

Table 5.6: Model A (float32) performance on the ESP32-S3.

| Win | Step | Stage A (ticks) | ELU-A (µs) | Stage B (ticks) | ELU-B (µs) | LSTM step (ticks) | Reg step (ticks) | Window (µs) | NMSE |
|---|---|---|---|---|---|---|---|---|---|
| 50 | 1 | 146 253 | 1 624 | 3 753 | 780.9 | 5 240 | 166.7 | 296 856 | $1.09\times10^{-10}$ |
| | 5 | 146 286 | 1 627 | 3 740 | 776.4 | 26 186 | 433.8 | 292 829 | $1.09\times10^{-10}$ |
| 100 | 1 | 316 995 | 3 707 | 9 175 | 1 550 | 5 208 | 166.5 | 621 029 | $1.09\times10^{-10}$ |
| | 5 | 316 958 | 3 655 | 9 248 | 1 547 | 26 133 | 442.1 | 614 409 | $1.09\times10^{-10}$ |
| 150 | 1 | 487 957 | 6 025 | 14 984 | 2 319 | 5 212 | 171.3 | 950 779 | $1.14\times10^{-10}$ |
| | 5 | 487 972 | 6 026 | 14 898 | 2 317 | 26 096 | 434.2 | 936 707 | $1.14\times10^{-10}$ |
| 200 | 1 | 658 629 | 8 104 | 20 351 | 3 184 | 5 205 | 165.8 | 1 275 203 | $1.06\times10^{-10}$ |
| | 5 | 658 625 | 8 115 | 20 360 | 3 179 | 26 124 | 437.0 | 1 259 205 | $1.06\times10^{-10}$ |
| 250 | 1 | 829 242 | 10 216 | 25 645 | 4 074 | 5 202 | 166.0 | 1 600 144 | $1.15\times10^{-10}$ |
| | 5 | 829 243 | 10 133 | 25 655 | 4 069 | 26 121 | 436.2 | 1 580 133 | $1.14\times10^{-10}$ |
| 300 | 1 | 999 930 | 12 137 | 30 688 | 4 927 | 5 213 | 166.6 | 1 926 506 | $1.14\times10^{-10}$ |
| | 5 | 999 927 | 12 140 | 30 703 | 4 946 | 26 076 | 437.0 | 1 900 597 | $1.14\times10^{-10}$ |
| 350 | 1 | 1 170 780 | 14 108 | 35 826 | 5 794 | 5 211 | 166.5 | 2 251 311 | $1.23\times10^{-10}$ |
| | 5 | 1 171 051 | 14 111 | 35 808 | 5 791 | 26 057 | 439.9 | 2 220 261 | $1.23\times10^{-10}$ |
| 400 | 1 | 1 341 961 | 16 130 | 40 878 | 6 637 | 5 212 | 166.2 | 2 576 174 | $1.11\times10^{-10}$ |
| | 5 | 1 342 205 | 16 131 | 40 887 | 6 633 | 26 074 | 432.4 | 2 540 645 | $1.11\times10^{-10}$ |
| 450 | 1 | 1 513 493 | 18 186 | 45 942 | 7 480 | 5 208 | 166.3 | 2 901 490 | $1.21\times10^{-10}$ |
| | 5 | 1 513 760 | 18 189 | 45 951 | 7 478 | 26 071 | 430.6 | 2 861 844 | $1.21\times10^{-10}$ |
| 500 | 1 | 1 685 149 | 20 198 | 51 176 | 8 314 | 5 209 | 167.7 | 3 229 980 | $1.29\times10^{-10}$ |
| | 5 | 1 685 457 | 20 198 | 51 067 | 8 311 | 26 065 | 431.6 | 3 185 315 | $1.29\times10^{-10}$ |

Table 5.7: Model A (float32) memory footprint on the ESP32-S3.

| | | Model size (kB) | | | | | Tensor arena (kB) | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | A | B | LSTM | Reg | Total | A | B | LSTM | Reg | Total |
| Win | Step | | | | | | | | | | |
| 50 | 1 | 14.73 | 5.34 | 217.10 | 8.21 | 245.38 | 85.92 | 24.87 | 8.86 | 1.38 | 121.02 |
| | 5 | 14.73 | 5.34 | 274.69 | 8.45 | 303.21 | 85.91 | 24.86 | 24.77 | 3.13 | 138.67 |
| 100 | 1 | 14.73 | 5.34 | 217.10 | 8.21 | 245.38 | 170.29 | 46.73 | 8.86 | 1.37 | 227.26 |
| | 5 | 14.73 | 5.34 | 274.69 | 8.45 | 303.21 | 170.29 | 46.73 | 24.77 | 3.13 | 244.93 |
| 150 | 1 | 14.73 | 5.34 | 217.10 | 8.21 | 245.38 | 254.67 | 68.61 | 8.86 | 1.37 | 333.51 |
| | 5 | 14.73 | 5.34 | 274.69 | 8.45 | 303.21 | 254.66 | 68.62 | 24.77 | 3.13 | 351.17 |
| 200 | 1 | 14.73 | 5.34 | 217.10 | 8.21 | 245.38 | 339.04 | 90.49 | 8.85 | 1.36 | 439.75 |
| | 5 | 14.73 | 5.34 | 274.69 | 8.45 | 303.21 | 339.04 | 90.49 | 24.77 | 3.13 | 457.43 |
| 250 | 1 | 14.73 | 5.34 | 217.10 | 8.21 | 245.38 | 423.42 | 112.36 | 8.85 | 1.36 | 546.00 |
| | 5 | 14.73 | 5.34 | 274.69 | 8.45 | 303.21 | 423.42 | 112.36 | 24.77 | 3.13 | 563.68 |
| 300 | 1 | 14.73 | 5.34 | 217.10 | 8.21 | 245.38 | 507.79 | 134.24 | 8.85 | 1.36 | 652.25 |
| | 5 | 14.73 | 5.34 | 274.69 | 8.45 | 303.21 | 507.79 | 134.24 | 24.77 | 3.13 | 669.93 |
| 350 | 1 | 14.73 | 5.34 | 217.10 | 8.21 | 245.38 | 592.17 | 156.11 | 8.85 | 1.36 | 758.50 |
| | 5 | 14.73 | 5.34 | 274.69 | 8.45 | 303.21 | 592.16 | 156.12 | 24.76 | 3.13 | 776.17 |
| 400 | 1 | 14.73 | 5.34 | 217.10 | 8.21 | 245.38 | 676.54 | 178.00 | 8.86 | 1.36 | 864.77 |
| | 5 | 14.73 | 5.34 | 274.69 | 8.45 | 303.21 | 676.53 | 178.00 | 24.77 | 3.13 | 882.43 |
| 450 | 1 | 14.73 | 5.34 | 217.10 | 8.21 | 245.38 | 760.92 | 199.87 | 8.86 | 1.37 | 971.02 |
| | 5 | 14.73 | 5.34 | 274.69 | 8.45 | 303.21 | 760.91 | 199.87 | 24.77 | 3.13 | 988.68 |
| 500 | 1 | 14.73 | 5.34 | 217.10 | 8.21 | 245.38 | 845.29 | 221.74 | 8.86 | 1.37 | 1077.26 |
| | 5 | 14.73 | 5.34 | 274.69 | 8.45 | 303.21 | 845.28 | 221.75 | 24.77 | 3.13 | 1094.93 |

Table 5.8: Model A (int8 A,B, ELU stages) performance on the ESP32-S3.

| Win | Step | Stage A (µs) | ELU-A (µs) | Stage B (µs) | ELU-B (µs) | LSTM step (µs) | Reg step (µs) | Window (µs) | NMSE |
|---|---|---|---|---|---|---|---|---|---|
| 50 | 1 | 8 989 | 125.6 | 987.0 | 771.4 | 5 296 | 166.8 | 156 982 | $6.06\times10^{-3}$ |
| | 5 | 8 957 | 125.6 | 993.9 | 771.9 | 26 315 | 439.5 | 151 647 | $6.06\times10^{-3}$ |
| 100 | 1 | 19 728 | 245.7 | 1 694 | 1 499 | 5 271 | 170.7 | 313 867 | $1.51\times10^{-2}$ |
| | 5 | 19 747 | 245.5 | 1 684 | 1 499 | 26 263 | 435.3 | 303 781 | $1.51\times10^{-2}$ |
| 150 | 1 | 29 840 | 365.8 | 2 474 | 2 159 | 5 269 | 169.2 | 472 762 | $9.26\times10^{-3}$ |
| | 5 | 29 865 | 365.6 | 2 496 | 2 161 | 26 223 | 432.0 | 455 287 | $9.26\times10^{-3}$ |
| 200 | 1 | 39 911 | 485.7 | 3 710 | 2 865 | 5 268 | 167.3 | 630 400 | $1.68\times10^{-2}$ |
| | 5 | 39 915 | 486.1 | 3 710 | 2 865 | 26 272 | 441.4 | 609 025 | $1.68\times10^{-2}$ |
| 250 | 1 | 49 905 | 605.6 | 5 029 | 3 677 | 5 266 | 167.4 | 788 276 | $4.98\times10^{-3}$ |
| | 5 | 49 907 | 605.8 | 5 029 | 3 677 | 26 263 | 440.4 | 761 536 | $4.98\times10^{-3}$ |
| 300 | 1 | 59 938 | 726.0 | 6 420 | 4 396 | 5 267 | 169.7 | 947 914 | $4.18\times10^{-3}$ |
| | 5 | 59 937 | 725.9 | 6 434 | 4 392 | 26 259 | 439.6 | 914 841 | $4.18\times10^{-3}$ |
| 350 | 1 | 69 966 | 847.4 | 8 202 | 5 431 | 5 271 | 168.4 | 1 108 081 | $9.29\times10^{-3}$ |
| | 5 | 69 647 | 848.2 | 8 154 | 5 429 | 26 201 | 436.6 | 1 065 835 | $9.29\times10^{-3}$ |
| 400 | 1 | 79 777 | 968.0 | 9 685 | 6 200 | 5 281 | 172.0 | 1 269 728 | $4.76\times10^{-3}$ |
| | 5 | 79 794 | 989.4 | 9 869 | 6 198 | 26 205 | 437.8 | 1 219 822 | $4.76\times10^{-3}$ |
| 450 | 1 | 89 986 | 1 129 | 11 432 | 6 930 | 5 272 | 170.7 | 1 427 619 | $7.22\times10^{-3}$ |
| | 5 | 90 025 | 1 130 | 11 423 | 6 933 | 26 206 | 446.4 | 1 373 823 | $7.22\times10^{-3}$ |
| 500 | 1 | 100 236 | 1 295 | 12 752 | 7 712 | 5 256 | 168.9 | 1 579 754 | $4.72\times10^{-3}$ |
| | 5 | 100 261 | 1 317 | 12 742 | 7 712 | 26 216 | 449.4 | 1 527 258 | $4.72\times10^{-3}$ |

Table 5.9: Model A (int8 A,B, ELU stages) footprint on the ESP32-S3.

| | | Model size (kB) | | | | | Tensor arena (kB) | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | A | B | LSTM | Reg | Total | A | B | LSTM | Reg | Total |
| Win | Step | | | | | | | | | | |
| 50 | 1 | 8.57 | 5.27 | 217.10 | 8.21 | 239.15 | 60.95 | 13.18 | 8.86 | 1.38 | 84.38 |
| | 5 | 8.57 | 5.27 | 274.69 | 8.45 | 296.97 | 60.95 | 13.18 | 24.77 | 3.13 | 102.02 |
| 100 | 1 | 8.57 | 5.27 | 217.10 | 8.21 | 239.15 | 119.55 | 22.74 | 8.86 | 1.37 | 152.51 |
| | 5 | 8.57 | 5.27 | 274.69 | 8.45 | 296.97 | 119.55 | 22.74 | 24.77 | 3.13 | 170.19 |
| 150 | 1 | 8.57 | 5.27 | 217.10 | 8.21 | 239.15 | 178.14 | 32.32 | 8.86 | 1.37 | 220.69 |
| | 5 | 8.57 | 5.27 | 274.69 | 8.45 | 296.97 | 178.13 | 32.32 | 24.77 | 3.12 | 238.35 |
| 200 | 1 | 8.57 | 5.27 | 217.10 | 8.21 | 239.15 | 236.73 | 41.88 | 8.85 | 1.38 | 288.84 |
| | 5 | 8.57 | 5.27 | 274.69 | 8.45 | 296.97 | 236.73 | 41.88 | 24.77 | 3.12 | 306.52 |
| 250 | 1 | 8.57 | 5.27 | 217.10 | 8.21 | 239.15 | 295.33 | 51.46 | 8.85 | 1.38 | 357.02 |
| | 5 | 8.57 | 5.27 | 274.69 | 8.45 | 296.97 | 295.33 | 51.46 | 24.77 | 3.12 | 374.69 |
| 300 | 1 | 8.57 | 5.27 | 217.10 | 8.21 | 239.15 | 353.92 | 61.02 | 8.85 | 1.38 | 425.17 |
| | 5 | 8.57 | 5.27 | 274.69 | 8.45 | 296.97 | 353.92 | 61.02 | 24.77 | 3.13 | 442.84 |
| 350 | 1 | 8.57 | 5.27 | 217.10 | 8.21 | 239.15 | 412.52 | 70.60 | 8.85 | 1.38 | 493.35 |
| | 5 | 8.57 | 5.27 | 274.69 | 8.45 | 296.97 | 412.52 | 70.61 | 24.76 | 3.13 | 511.02 |
| 400 | 1 | 8.57 | 5.27 | 217.10 | 8.21 | 239.15 | 471.11 | 80.17 | 8.85 | 1.36 | 561.50 |
| | 5 | 8.57 | 5.27 | 274.69 | 8.45 | 296.97 | 471.11 | 80.16 | 24.77 | 3.13 | 579.17 |
| 450 | 1 | 8.57 | 5.27 | 217.10 | 8.21 | 239.15 | 529.71 | 89.75 | 8.86 | 1.37 | 629.68 |
| | 5 | 8.57 | 5.27 | 274.69 | 8.45 | 296.97 | 529.70 | 89.74 | 24.77 | 3.13 | 647.34 |
| 500 | 1 | 8.57 | 5.27 | 217.10 | 8.21 | 239.15 | 588.30 | 99.31 | 8.86 | 1.37 | 697.84 |
| | 5 | 8.57 | 5.27 | 274.69 | 8.45 | 296.97 | 588.29 | 99.30 | 24.77 | 3.13 | 715.50 |

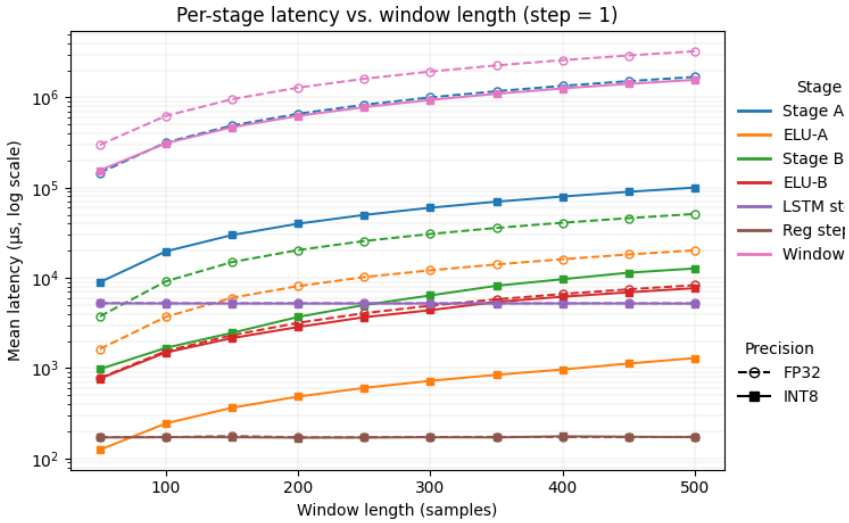



Figure 5.5: Figure shows latency of model stages over window length.

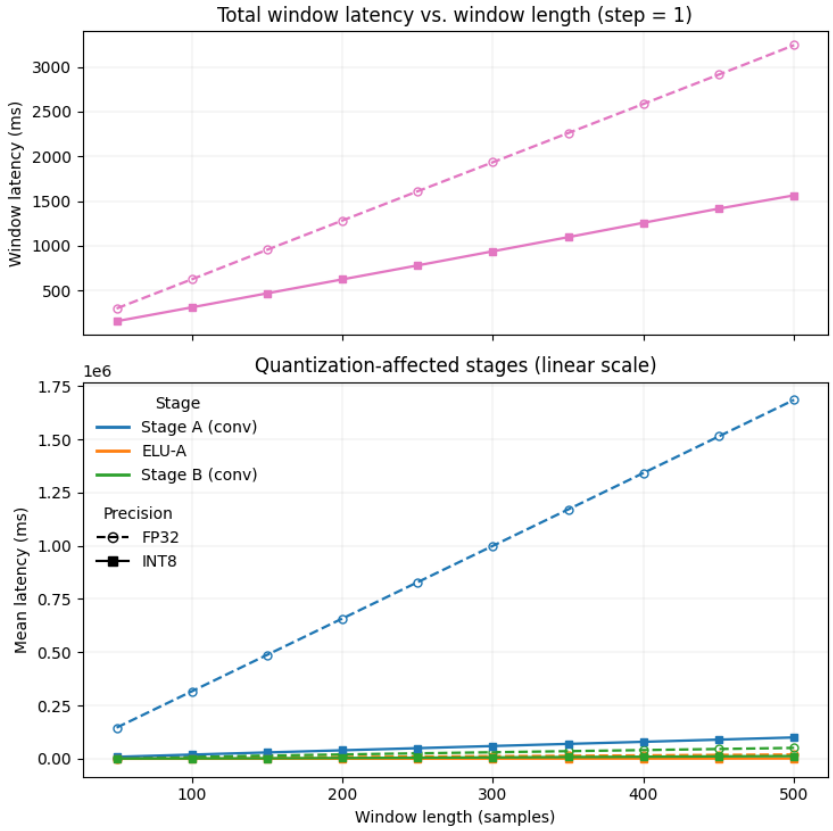



Figure 5.6: Figure shows latency of model stages over window length.

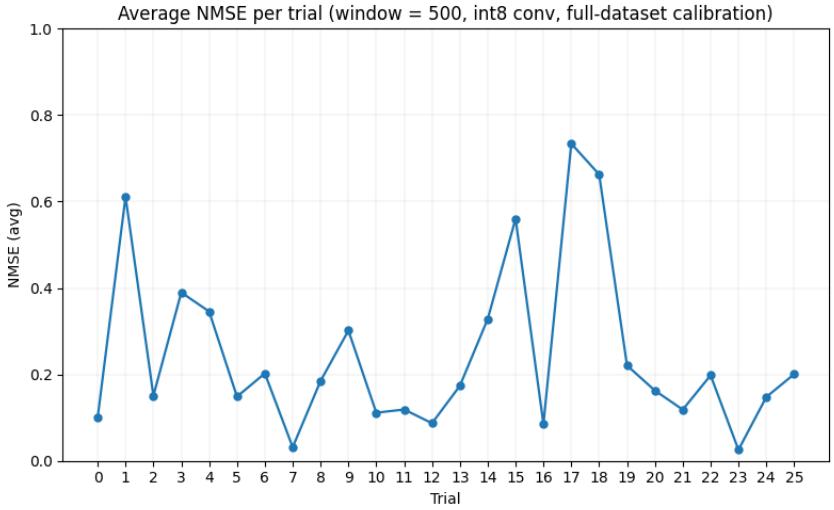



Figure 5.7: Average NMSE for the selected training-set trials under full-dataset calibration.

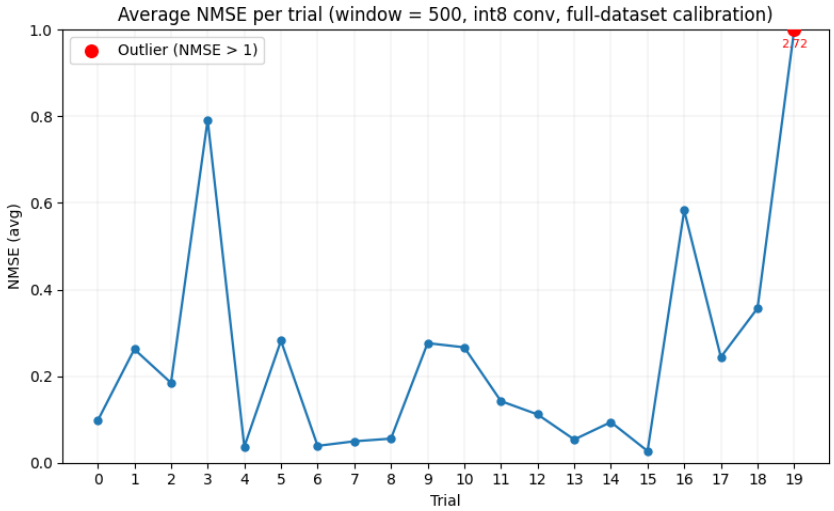



Figure 5.8: Average NMSE for the selected test-set trials under full-dataset calibration.

# 6 Discussion

## 6.1 Model Performance

### 6.1.1 Convolutional Stage Quantization

The results confirm that `int8` quantization yields substantial latency reductions for the convolutional inference stages on the ESP32-S3. Comparing the fully floating-point configuration (Table 5.6) with the `int8` configuration (Table 5.8), the Stage A inference cost falls from approximately 146,253 to 8,989 micro-seconds at a window length of 50, a speed-up of roughly 16×, while Stage B improves by a factor of about 3.8×. This disparity reflects the substantial gap between the integer arithmetic units and the floating-point unit on the ESP32-S3, and is further attributable to the ESP-NN backend, whose assembly-optimized kernels on this platform target `int8` operations exclusively, leaving floating-point paths to the generic, unoptimized implementations.

### 6.1.2 Numerical Fidelity Under Quantization

As expected, quantization introduces a measurable degradation in numerical fidelity. The floating-point model attains a consistent NMSE of approximately $1.09 \times 10^{-10}$, a value attributable to floating-point rounding noise rather than algorithmic error.

We first assess quantization fidelity under trial-specific calibration, in which the quantization parameters are calibrated on the same trial used for evaluation. This setting is diagnostic: it isolates the degradation intrinsic to `int8` representation under near-ideal calibration, before the additional error introduced by generalisation is considered. It is worth noting that a single trial comprises approximately 9,000 samples, so the calibration statistics are drawn from a large and representative span of the signal's dynamic range rather than a small sample. Under this trial-specific calibration of the convolutional stages, the NMSE rises to the range of approximately $4.2 \times 10^{-3}$ to $1.7 \times 10^{-2}$, corresponding to roughly 98 to 99.6% of the reference signal variance being preserved. Although this is several orders of magnitude above the floating-point baseline, the error remains small in absolute terms.

Calibrating instead on the entire training set and evaluating across multiple trials, uniformly picked from the set, yields a more demanding and realistic assessment, as a single set of quantization parameters must now generalise across recordings rather than being fitted to the trial under test. Figure 5.7 reports the resulting average NMSE on the training-set trials. Approximately 70% of trials attain an average NMSE at or below 0.2, and about 85% remain at or below 0.4. Four trials lie above 0.5, including a single pronounced outlier whose average NMSE exceeds unity.

Crucially, no evidence of systematic degradation is found across the held-out test trials. Figure 5.8 reports the average NMSE on the test-set trials, uniformly picked from the set. More

than half attain an average NMSE at or below 0.2, and about 85% remain below 0.4, with the four largest values reaching only approximately 0.56 to 0.73; with one test trial exceeding 1.0.

A closer inspection of the per-window errors reveals that even these higher trial-level averages are not representative of the typical window, but are instead dominated by a small number of outliers. Because the reported NMSE for a trial is obtained by averaging the NMSE computed independently over each window of the recording, a few windows exhibiting pronounced transient errors, "hiccups" in the reconstructed signal, disproportionately inflate the trial average. A representative example is the last trial in the training set whose average NMSE is approximately 0.20. The overwhelming majority of its windows attain NMSE values between roughly $10^{-4}$ and $10^{-2}$, indicating near-faithful reconstruction; the average is instead driven almost entirely by a single anomalous window whose NMSE reaches 2.34, accompanied by two further elevated windows at 0.65 and 0.31. That single worst window alone contributes approximately 0.13 to the mean, accounting for roughly two-thirds of the trial's average, despite the remaining windows being reconstructed faithfully.

Taken together, these results suggest that the degradation introduced by `int8` quantization of the convolutional stages remains tolerable given the magnitude of the accompanying latency improvements, particularly as the worst-case errors are confined to isolated windows rather than distributed across the signal, and as the error distribution is otherwise preserved between the training and test sets.

### 6.1.3 ELU Activation Handling

The manual `int8` look-up-table reimplementation of the first ELU block (Section 4.7.3) yields a substantial latency reduction. For this block (ELU-A), the substitution reduces the latency from approximately 1,624 to 125 µs at a window length of 50, and from 20,198 to 1,295 µs at a window length of 500, a speed-up of roughly 13 to 16×. As expected, the second activation block (ELU-B), retained in floating point, exhibits no comparable improvement; its latency is essentially unchanged between the two configurations (for example, 771 versus 780 µs at a window length of 50). This confirms that the cost of the decomposed ELU island stemmed from the floating-point quantize/dequantize traversals rather than the activation arithmetic itself, and that a single-pass LUT removes that cost almost entirely.

### 6.1.4 LSTM Step Size

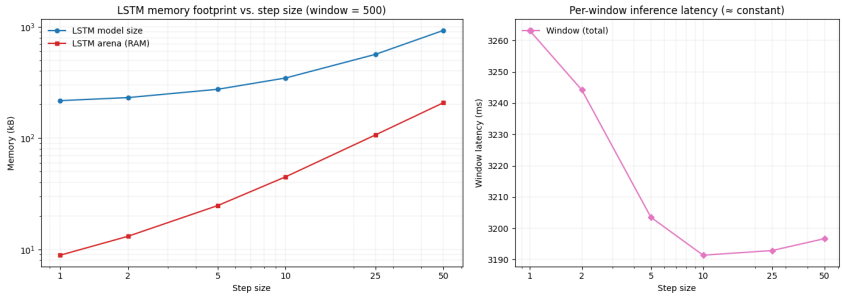



Figure 6.1: LSTM memory footprint and per-window inference latency as a function of step size, at a fixed window length of 500 samples. Memory (left, log scale) grows steeply with step size for both the model and the activation arena, whereas the per-window latency (right) remains essentially constant, varying by under 3% across the full range. Note that the latency axis is zoomed; the apparent dip corresponds to an absolute change of only a few tens of microseconds.

The effect of the LSTM step size (Section 4.7.2) was characterised by sweeping it from 1 to 50 at a fixed window length of 500 samples. The results (Figure 6.1, float32) reveal a pronounced asymmetry between memory and latency. The memory cost grows steeply with the step size: the LSTM model size rises from roughly 200 kB at a step of 1 to approximately 1,000 kB at a step of 50, while the activation arena grows even more sharply, from under 10 kB to about 200 kB, an increase of more than twentyfold. The per-window latency, by contrast, is essentially flat, remaining within a narrow band of approximately 3,190 to 3,260 ms across all step sizes; it decreases only marginally as the step size grows, since a larger step invokes the recurrent cell fewer times per window, but this saving plateaus almost immediately.

The practical implication is clear: increasing the step size yields no meaningful latency benefit while incurring a substantial and rapidly growing memory cost. The growth in arena size is the more critical of the two, as it directly constrains deployability; a sufficiently large arena may no longer fit in fast on-chip SRAM and must instead be allocated in slower external PSRAM, itself introducing additional memory-access latency. The step size should therefore be kept small in memory-constrained deployments, with a step of 1 offering the most favourable operating point: the lowest memory footprint with no latency penalty.

### 6.1.5 Window Length

Our results show a near-linear relationship between the window length and the per-window inference latency: in the floating-point configuration, the mean per-window latency rises from approximately 156,982 µs at a window length of 50 to 1,579,754 µs at 500 Table 5.8, an essentially constant per-sample cost of roughly 3,000 µs.

This growth does not, however, translate into a proportional change in the time required to process a complete trial. Because each trial has a fixed length and the windows are non-overlapping, a longer window is offset by proportionally fewer windows per trial, so the two effects cancel and the total work per trial is approximately invariant to the window length. At the level of trial

throughput the choice of window length is therefore largely neutral, and the per-window figures above should not be read as an end-to-end cost that grows with window size.

The window length nonetheless matters for deployment by way of the sampling frequency. The number of samples in a window is set jointly by the temporal span it covers and the rate at which the signal is sampled. For a fixed real-time span, for instance the 250 ms commonly adopted for myoelectric control, the sample count, and hence the per-window cost, scales directly with the sampling rate. The NinaPro DB2 recordings used here are sampled at 2 kHz, so a 250 ms window comprises 500 samples, the largest configuration evaluated. Sampling the same span at 1 kHz would halve the count to 250 and, by the near-linear relationship above, roughly halve the per-window latency, as borne out by the measured drop from 1,579,754 µs at 500 samples to 788,276 µs at 250. Subject to the signal-fidelity constraints of the application, the sampling frequency is thus a direct means of improving latency budget without altering the window span or the model architecture.

### 6.1.6 Quantization of the Recurrent and Regression Stages

Finally, our results indicate that quantization of the LSTM and regression stages is counterproductive, offering minimal returns relative to the implementation difficulty. Unlike feed-forward architectures such as dense and convolutional layers, the LSTM is recurrent: its output at a given timestep feeds the computation at the next, which causes quantization error to compound across timesteps, so that even small errors at the initial timestep accumulate into substantial deviations at the final output. The same effect applies to the regression head, which is invoked within the same recurrent loop. Cell-state quantization is particularly problematic. In contrast to the hidden state, the cell state exhibits a wide dynamic range, spanning approximately $-30$ to 30 in our experiments, which causes standard `int8` quantization to be coarse: numerically significant variations are collapsed into narrow ranges, producing repeated saturated values and severely degraded error metrics. The conventional remedy is a dedicated `int16` quantization scheme for the cell-state path, which is supported neither by the LiteRT converter nor by the ESP-NN kernels. This would require hand-written operations for the elementwise multiplications and additions, together with more memory-intensive LUTs for activations such as tanh (64 kB each, compared with 256 bytes for `int8` data). The literature on LSTM quantization remains sparse, with the few available studies recommending quantization-aware training methods, which lie beyond the scope of this work.

## 6.2 Limitations

Several limitations of this work merit discussion.

First, the pipeline did not achieve the real-time inference latency defined in the literature. Our most optimised configuration required approximately 1,500 ms per window at a window length of 500 samples, and approximately 150 ms at a window length of 50 samples. The latter figure is, in fact, within the 100–300 ms upper bound that the literature identifies as the maximum tolerable end-to-end latency for closed-loop myoelectric control, though it lies at the upper edge of the 50–150 ms range associated with fluid and intuitive interaction; the former, by contrast, exceeds both bounds by an order of magnitude. During the experiments we attempted a platform-specific implementation of the two-layer LSTM, but it produced numerically inaccurate outputs and was

therefore discontinued within the scope of this thesis. Notably, even this implementation, setting its accuracy aside, reduced the per-step LSTM latency from 5.2 ms to 3 ms and the total window latency to 942 ms. This indicates that even had the optimised LSTM been numerically reliable, sub-100 – 300 ms latency would still have been unattainable at the standard window length, locating the latency bottleneck in the model architecture itself rather than in the quantisation or implementation strategy.

Second, the scope of this thesis was confined to the implementation of the provided model architecture, which was adopted without modification. This is a relevant consideration, as it has been argued that purely convolutional (TCN) architectures can be more reliable than recurrent (RNN) architectures for capturing temporal dependencies in sEMG data [18]; a systematic comparison between the two paradigms on the target hardware therefore remains outside the scope of the present work.

Third, we deliberately adopted a cross-platform approach, seeking to avoid tying the pipeline to any single hardware target. Although all experiments were conducted on ESP32 chips, the pipeline is readily generalisable, as it relies on the industry-standard LiteRT framework. While LiteRT and TensorFlow Lite Micro map operations to native kernel backends, the final performance could nonetheless benefit substantially from platform-specific optimisations that this work did not pursue.

Fourth, although several LSTM quantisation schemes were investigated, none yielded reliable results. This outcome is attributable to a number of factors that warrant further investigation before faster and more dependable quantisation of recurrent layers can be achieved.

Finally, because the model was supplied pre-trained, quantisation-aware training (QAT) was not explored in depth. Several studies nonetheless report considerable benefits of QAT over simpler post-training quantisation (PTQ) schemes, suggesting a promising direction for improvement. Additionally, alternative quantisation schemes such as hybrid and int16 quantisation were not examined, owing to their incompatibility with the Espressif platforms targeted in this work.

# 7 Conclusion

## 7.1 Future Work

The limitations identified above suggest several promising directions for future research.

First, because the latency bottleneck is architectural, achieving real-time inference will require modifications to the model itself. Promising candidates include increasing the downsampling factor, reducing the LSTM to a single layer, replacing the LSTM with the more lightweight GRU, or removing the recurrent module altogether. Such changes would reduce the end-to-end latency and may bring it within the real-time budget.

Second, the deployment pipeline could be evaluated across a broader range of model architectures. In particular, a systematic comparison between the recurrent (LSTM-based) model studied here and a purely convolutional (TCN) counterpart, conducted directly on the target hardware, would clarify the trade-offs between accuracy, memory footprint, and inference latency for each paradigm under realistic embedded constraints.

Third, while the cross-platform design of the pipeline favours portability, future work could quantify the performance gains attainable through platform-specific optimisation. Deploying the pipeline on hardware with dedicated neural-network acceleration, or substituting hand-optimised kernels for the default backend, would establish how much additional efficiency can be recovered without sacrificing the framework's generality.

Fourth, the difficulties encountered in quantising the recurrent layers motivate a dedicated investigation into reliable quantisation of LSTM and other RNN-based components. Identifying the sources of instability, and developing or adopting quantisation strategies tailored to recurrent computation, would directly address one of the central implementation challenges of this work.

Finally, the quantisation methodology itself could be extended. Incorporating quantisation-aware training (QAT), provided the training pipeline is made available, may yield accuracy improvements over the post-training quantisation employed here. Likewise, evaluating alternative schemes such as hybrid and int16 quantisation on hardware platforms that support them would help characterise the full design space of accuracy-versus-efficiency trade-offs for embedded sEMG regression.

## 7.2 Conclusion

This thesis investigated the feasibility of deploying a convolutional–recurrent sEMG gesture-regression model on commodity, resource-constrained microcontroller hardware using a portable, non-destructive deployment methodology. Taking a pre-trained PyTorch network as given, we adapted it for embedded inference on the ESP32-S3 through the industry-standard LiteRT framework, without altering its architecture or retraining its parameters.

Our central contribution is the modular deployment pipeline. By partitioning the network into four independently exportable stages and exporting the LSTM as a windowed block invoked repeatedly within an on-device loop rather than unrolled across the full sequence, we reduced the model footprint from over 2.8 MB to approximately 260 KB and enabled mixed-precision deployment. Two graph-level transformations, folding batch normalization into preceding convolutions and reformulating Conv1D as Conv2D, were necessary to obtain correct and efficient quantized stages.

Our empirical results establish an asymmetric picture of where quantization pays off. For the convolutional stages, int8 quantization delivered substantial gains (up to 16× lower latency), driven by the ESP-NN backend's assembly-optimized int8 kernels, at a tolerable fidelity cost (the quantized model retained roughly 98–99.6% of the reference signal variance); replacing the unsupported ELU activation with an int8 look-up table yielded a further 13–16× speed-up. For the recurrent and regression stages, by contrast, quantization proved counterproductive: intermittent requantization dominates latency, recurrent feedback compounds quantization error across timesteps, and the wide dynamic range of the cell state defeats coarse int8 representation, with the conventional int16 remedy supported by neither LiteRT nor ESP-NN.

Taken together, these findings characterise both the promise and the limits of portable, framework-based TinyML deployment. The approach makes accurate sEMG regression feasible on cheap, widely available hardware without hardware-model co-design, while exposing the recurrent layer as the principal remaining obstacle, one rooted in current framework and kernel support rather than in any fundamental barrier.

# List of Figures

# List of Tables

# Bibliography


[1] M. Zheng, M. Crouch and M. Eggleston, "Surface Electromyography as a Natural Human-Machine Interface: A Review," *IEEE Sensors Journal*, vol. 22, pp. 1–1, May 2022.

[2] I. V. Valcheva, V. Gandhi and E. Chinellato, "Surface EMG driven gesture recognition using machine learning for robotic applications," *Discover Robotics*, vol. 2, no. 1, p. 4, Feb. 2026.

[3] S. Ni, M. A. A. Al-qaness, A. Hawbani, D. Al-Alimi, M. Abd Elaziz and A. A. Ewees, "A survey on hand gesture recognition based on surface electromyography: Fundamentals, methods, applications, challenges and future trends," *Applied Soft Computing*, vol. 166, p. 112235, Nov. 2024.

[4] S. Salminger, H. Stino, L. H. Pichler, C. Gstoettner, A. Sturma, J. A. Mayer, M. Szivak and O. C. Aszmann, "Current rates of prosthetic usage in upper-limb amputees - have innovations had an impact on device acceptance?" *Disability and Rehabilitation*, vol. 44, no. 14, pp. 3708–3713, Jul. 2022.

[5] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang and S. Han, "Tiny Machine Learning: Progress and Futures," 2024.

[6] H. Yelchuri and R. R, "A review of TinyML," 2022.

[7] R. Martinek, M. Ladrova, M. Sidikova, R. Jaros, K. Behbehani, R. Kahankova and A. Kawala-Sterniuk, "Advanced Bioelectrical Signal Processing Methods: Past, Present and Future Approach—Part I: Cardiac Signals," *Sensors*, vol. 21, no. 15, p. 5186, Jul. 2021.

[8] Jonas Koellner and Bin Yang, "Towards natural wrist-hand motion: A lightweight neural network for proportional EMG-driven prosthetic control."

[9] M. Raez, M. Hussain and F. Mohd-Yasin, "Techniques of EMG signal analysis: Detection, processing, classification and applications," *Biological Procedures Online*, vol. 8, pp. 11–35, Mar. 2006.

[10] U. Côté Allard, C. L. Fall, A. Drouin, A. Campeau-Lecours, C. Gosselin, K. Glette, F. Laviolette and B. Gosselin, "Deep Learning for Electromyographic Hand Gesture Signal Classification Using Transfer Learning," *IEEE transactions on neural systems and rehabilitation engineering: a publication of the IEEE Engineering in Medicine and Biology Society*, vol. PP, Jan. 2019.

[11] C. J. De Luca, L. Donald Gilmore, M. Kuznetsov and S. H. Roy, "Filtering the surface EMG signal: Movement artifact and baseline noise contamination," *Journal of Biomechanics*, vol. 43, no. 8, pp. 1573–1579, May 2010.

[12] B. Hudgins, P. Parker and R. Scott, "A new strategy for multifunction myoelectric control," *IEEE Transactions on Biomedical Engineering*, vol. 40, no. 1, pp. 82–94, Jan. 1993.

[13] L. H. Smith, L. J. Hargrove, B. A. Lock and T. A. Kuiken, "Determining the Optimal Window Length for Pattern Recognition-Based Myoelectric Control: Balancing the Competing Effects of Classification Error and Controller Delay," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 19, no. 2, pp. 186–192, Apr. 2011.

[14] A. Phinyomark, P. Phukpattaranont and C. Limsakul, "Feature Reduction and Selection for EMG Signal Classification," *Expert Systems with Applications*, vol. 39, pp. 7420–7431, Jun. 2012.

[15] U. Côté-Allard, E. Campbell, A. Phinyomark, F. Laviolette, B. Gosselin and E. Scheme, "Interpreting Deep Learning Features for Myoelectric Control: A Comparison with Handcrafted Features," *Frontiers in Bioengineering and Biotechnology*, vol. 8, p. 158, Mar. 2020.

[16] P. T., V. K. Elumalai and B. E., "Hand gesture classification framework leveraging the entropy features from sEMG signals and VMD augmented multi-class SVM," *Expert Systems with Applications*, vol. 238, p. 121972, Mar. 2024.

[17] N. Mohapatra, J. Sahoo and G. K. Sahoo, *Hand Gesture Classification Using Surface Electromyography Signals Based on Fusion of Time Domain Features*, Apr. 2024.

[18] M. Zanghieri, S. Benatti, A. Burrello, V. Kartsch, F. Conti and L. Benini, "Robust Real-Time Embedded EMG Recognition Framework Using Temporal Convolutional Networks on a Multicore IoT Processor," *IEEE Transactions on Biomedical Circuits and Systems*, vol. 14, no. 2, pp. 244–256, Apr. 2020.

[19] W. Zhong, X. Jiang, K. Szymaniak, M. Jabbari, C. Ma and K. Nazarpour, "Deep Feature Learning From Electromyographic Signals for Gesture Recognition Systems," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 34, pp. 20–35, 2026.

[20] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan and S. Han, "MCUNet: Tiny Deep Learning on IoT Devices," 2020.

[21] R. David, J. Duke, A. Jain, V. J. Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regev, R. Rhodes, T. Wang and P. Warden, "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems," Mar. 2021.

[22] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," Jun. 2018.

[23] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam and D. Kalenichenko, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," Dec. 2017.

[24] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen and T. Blankevoort, "A White Paper on Neural Network Quantization," Jun. 2021.

[25] A. Faraone and R. Delgado, "Convolutional-Recurrent Neural Networks on Low-Power Wearable Platforms for Cardiac Arrhythmia Detection," p. 157, Aug. 2020.

[26] E. Kim, J. Kim, J. Park, H. Ko and Y. Kyung, "TinyML-Based Classification in an ECG Monitoring Embedded System," *Computers, Materials & Continua*, vol. 75, pp. 1751–1764, Jan. 2023.

[27] M. Zanghieri, S. Benatti, A. Burrello, V. J. Kartsch Morinigo, R. Meattini, G. Palli, C. Melchiorri and L. Benini, "sEMG-based Regression of Hand Kinematics with Temporal Convolutional Networks on a Low-Power Edge Microcontroller," in *2021 IEEE International*

*Conference on Omni-Layer Intelligent Systems (COINS)*. Barcelona, Spain: IEEE, Aug. 2021, pp. 1–6.

[28] M. Atzori, A. Gijsberts, I. Kuzborskij, S. Elsig, A.-G. M. Hager, O. Deriaz, C. Castellini, H. Muller and B. Caputo, "Characterization of a benchmark database for myoelectric movement classification," *IEEE transactions on neural systems and rehabilitation engineering: a publication of the IEEE Engineering in Medicine and Biology Society*, vol. 23, no. 1, pp. 73–83, Jan. 2015.

[29] "Ninapro Official Website," https://ninapro.hevs.ch/.

[30] Espressif Systems, "ESP32-S3 Technical Reference."

## Declaration

Herewith, I declare that I have developed and written the enclosed thesis entirely by myself and that I have not used sources or means except those declared.

This thesis has not been submitted to any other authority to achieve an academic grading and has not been published elsewhere.

Stuttgart, TBD Date of sign. Andrew Fady Fahmy